TRUSTEANDO

A Decentralised Verifiable Knowledge Graph Protocol

confidencenode.org/protocolos/trusteando

Autor: confidencenode.org/members/confidencenode0

18 de Marzo de 2026 — v0.1

Abstract

Trusteando is an open protocol for a decentralised, cryptographically verifiable Knowledge Graph. Its core is a class with five functions — fewer than twenty lines of code — from which the entire system emerges:

```
class TrusteandoNode:
    def __init__(self, key): self.key = key
    @staticmethod
    def reduce_hash(seed, elements):
        for e in elements: seed = hash(seed + e)
        return seed
    def grant_key(self, child):
        return hash(self.key + child)
    def respond_to_challenge(self, ctx):
        return self.reduce_hash(self.key, ctx)
    def verify_child_authorship(self, child, ctx, proof):
        return self.reduce_hash(self.grant_key(child), ctx) == proof
```

From this simplicity the entire protocol emerges: identity, authentication, authorisation, hierarchies, privacy and portability. Nodes are entities identified by the hash of their public URL or by an autonomously generated identifier. Relations are expressed as folder structures published by each entity in its own web space — the folder structure is the credential. Trust chains are established via hierarchically derived keys: each node receives its key from its parent via grant_key, and any interaction is proven with respond_to_challenge without revealing keys. The graph is dynamic with full verifiable history via since/until and plan/execution conventions. Identity is portable via an emergency key mechanism and old_identities tables published by receiving entities. The system is polycentric by design: any sufficiently reputable node can act as a root by executing the same public algorithm, without requiring blockchain. When adopted by public administrations, the folder schema acts as a universal interoperability contract: any system that exposes its data respecting the schema becomes interoperable by construction with any other, without bilateral agreements or intermediary platforms. Membership precedes recognition. Contextual trust suffices for most verification. Cryptographic verification handles the rest.

---

1. Introducción

Los sistemas de identidad digital existentes comparten un defecto estructural: requieren que la identidad sea concedida por una autoridad central antes de poder existir. Los certificados X.509 dependen de Autoridades de Certificación comerciales. Los sistemas de identidad federada como OAuth delegan la identidad a proveedores privados — Google, Apple, Microsoft — que pueden revocarla unilateralmente. Los Knowledge Graphs existentes como Wikidata o Google Knowledge Graph son centralizados o requieren consenso de un editor central para escribir.

Trusteando parte de una observación diferente: la identidad de una entidad real no necesita ser concedida — ya existe. Una universidad existe. Tiene una URL. Tiene personal. Tiene autoridad real sobre ese personal. Las relaciones entre entidades — quién pertenece a qué, desde cuándo, con qué rol — también existen. Lo que falta no es un mecanismo para crear esa identidad sino un protocolo para expresarla como un grafo de conocimiento verificable, descentralizado, y con memoria temporal.

Trusteando es ese protocolo. Los nodos del grafo son entidades identificadas por el hash de su URL pública. Las aristas son relaciones expresadas como estructuras de carpetas publicadas por cada entidad en su propio espacio web. La firma criptográfica de cada entidad sobre su propio espacio garantiza la integridad. La cadena de confianza entre niveles permite la verificación sin depender de ningún servidor central en el uso cotidiano.

1.1 Una red paralela sobre la misma web

Trusteando no compite con la web — se superpone a ella. Cada entidad que publica una carpeta trusteando/ en su dominio añade un nodo a una red paralela que corre sobre la misma infraestructura HTTP, los mismos servidores, las mismas URLs. La diferencia es lo que esa carpeta contiene: no HTML para consumo humano sino estructuras de carpetas que expresan hechos verificables criptográficamente.

La web dice: existe este documento. Trusteando dice: existe esta relación, y está verificada por quien tiene autoridad sobre ella. Esa distinción permite representar con precisión cualquier estructura organizativa — universidades con sus facultades, empresas con sus departamentos, estados con sus administraciones — y controlar de forma granular quién puede certificar qué sobre quién. La jerarquía de carpetas refleja la jerarquía real, y la criptografía garantiza que solo quien tiene autoridad puede emitir credenciales en su ámbito.

Ambas capas usan la misma infraestructura. Solo cambian las reglas.

2. Principios de diseño

2.1 La URL como ancla de identidad

El identificador de cualquier entidad en el sistema es el hash criptográfico de su URL canónica:

```
entity_id = SHA-256(canonical_url)
```

La entidad no necesita registrarse en ningún sistema previo. Su URL ya existe y es pública. Trusteando la reconoce como ancla de identidad sin añadir fricción. Dos entidades distintas tienen distintas URLs y por tanto distintos identificadores — no hay colisión posible bajo SHA-256 en la práctica.

La URL canónica se define como la URL de la página principal de la entidad con esquema https, sin barra final, en minúsculas. Por ejemplo: https://universidad.es

2.2 La pertenencia precede al reconocimiento

Si una universidad publica en su propia web — por ejemplo en https://universidad.es/trusteando — el hash de un profesor junto con su clave pública, ese profesor ya pertenece al espacio de identidad de esa universidad. La cadena de confianza existe antes de que el nodo raíz la valide formalmente.

El nodo raíz no crea la relación entre la universidad y su profesor — la eleva a confianza criptográficamente verificable por terceros que no conocen a ninguna de las partes. Para el verificador que ya conoce y confía en la universidad, la publicación en su URL es suficiente.

2.3 La confianza contextual es suficiente para el uso cotidiano

Ver el hash de Juan Escobar publicado en universidad.es/profesores ya transmite confianza a cualquier verificador que conozca esa universidad. No hace falta que el nodo raíz lo certifique. La universidad tiene autoridad real — su URL es su identidad, su reputación institucional es su aval.

La verificación criptográfica completa resuelve los casos donde la confianza contextual no alcanza: cuando el verificador no conoce la entidad, cuando la verificación debe ser automática, o cuando hay disputa sobre la autenticidad de una credencial.

2.5 La autoridad se demuestra controlando la URL

La autoridad de una entidad para emitir credenciales no se declara — se demuestra controlando la URL bajo la que publica. Nadie puede publicar bajo universidad.es/profesores/ sin controlar universidad.es. El DNS y el servidor web son la prueba de control, exactamente igual que en HTTPS.

Esto resuelve la pregunta de cómo sabe un verificador que una entidad tiene autoridad para emitir una credencial: porque la credencial está publicada bajo una URL que esa entidad controla. No hace falta ninguna declaración adicional. El control es la autoridad.

```
confidencenode.org                                  ← ConfidenceNode
controla esto
  /protocolos/trusteando                            ← por tanto controla
esto
  /members/confidencenode0                         ← y esto
  /members/confidencenode0/externals/github
    → github.com/ConfidenceNode0                   ← presencia externa
vinculada

universidad.es                                      ← la universidad
controla esto
  /profesores/juan-ruiz                            ← por tanto controla
esto
  /old_identities/                                 ← y esto
```

La carpeta externals permite a una entidad vincular desde su espacio propio cualquier presencia en sistemas externos que no controla — GitHub, ORCID, LinkedIn, rankings, registros oficiales. El vínculo lo firma la propia entidad, así que la autoridad sobre el vínculo es suya. Pero externals es semántica del protocolo, no una convención de nomenclatura: declara explícitamente que la entidad no tiene autoridad sobre lo que hay en el destino. Un verificador

que lee externals/github sabe que la entidad vincula algo que no controla y que por tanto no puede garantizar su contenido — solo garantiza que el vínculo es auténtico.

Esta distinción contrasta directamente con el resto de carpetas, donde la ausencia de externals implica control:

```
/members/confidencenode0/externals/github
  → la entidad declara que NO controla el destino

/protocolos/trusteando
  → la entidad declara implícitamente que SÍ controla esto

Otros ejemplos válidos:
  /members/confidencenode0/externals/orcid
  /members/confidencenode0/externals/linkedin
  universidad.es/externals/ministerio-educacion
  universidad.es/externals/rankings/qs-world
```

En Trusteando la estructura de la URL declara la autoridad. La carpeta externals es la única excepción explícita — y su existencia hace que la regla general sea aún más clara.

Este mecanismo ilustra un caso real: confidencenode.org/members/confidencenode0/externals/github apunta a github.com/ConfidenceNode0, donde vive el código del protocolo. El dominio propio es la identidad canónica; GitHub es el hosting actual del código. Cuando el hosting propio esté levantado, el código migrará — pero la identidad canónica no cambia.

2.6 La regla de oro — la estructura de carpetas es la credencial

Toda la información que una entidad está obligada a ofrecer sobre credenciales está almacenada en su estructura de carpetas públicas. No hay base de datos oculta, no hay campos en documentos separados, no hay APIs con estado interno que un verificador necesite consultar.

Un verificador puede comprobar cualquier credencial siguiendo únicamente URLs públicas. Si la información no está en la estructura de carpetas, no existe como credencial del protocolo. Esta regla no es una convención — es una condición de validez.

Las consecuencias prácticas son profundas: verificabilidad total sin cooperación de la entidad emisora, auditabilidad completa del espacio de identidad de cualquier entidad, y resiliencia ante caídas — una copia estática de las carpetas es suficiente para reconstruir toda la información.

2.7 Trusteando como Knowledge Graph descentralizado

El conjunto de entidades y relaciones que el protocolo expresa forma un Knowledge Graph con tres propiedades que los sistemas existentes no combinan:

- Descentralizado — cada nodo escribe en su propio espacio. Nadie necesita permiso de un editor central para publicar relaciones.
- Verificable criptográficamente — cada arista está firmada por quien la emite. No hay que confiar en ningún editor central — la firma lo prueba.
- Dinámico con historial completo — las carpetas since/until dan dimensión temporal al grafo. No es una foto estática sino un grafo que evoluciona con historia íntegra y verificable.

Las credenciales son aristas del grafo. Las entidades son nodos. La cadena de confianza es la topología. El protocolo de identidad es un caso particular de este grafo más general.

2.8 El protocolo es abierto e irrevocablemente público

El formato del protocolo, la capacidad de montar nodos propios, y la verificación manual consultando URLs directamente son libres y permanecen así. Ninguna entidad — incluido el nodo raíz de Trusteando — puede impedir a nadie implementar el protocolo, verificar credenciales, o publicar su propio espacio de identidad.

Esta apertura no es una concesión — es lo que da valor al sistema. Un protocolo que puede ser revocado por su creador no es infraestructura: es un servicio con condiciones de uso.

2.9 Interoperabilidad por diseño

Una consecuencia estructural de anclar la identidad en URLs públicas y expresar las relaciones como un esquema de carpetas es que cualquier sistema — base de datos, API REST, microservicio — que exponga sus datos respetando ese esquema se vuelve interoperable por defecto con cualquier otro que haga lo mismo. No hace falta negociar convenios bilaterales ni desarrollar conectores específicos: la estructura de carpetas es el contrato de interoperabilidad. Esto tiene implicaciones especialmente relevantes para administraciones públicas y organismos, donde la interoperabilidad actual exige plataformas intermediarias complejas y acuerdos punto a punto. El caso de uso se desarrolla en la sección 7.8.

2.10 El grafo como registro histórico inmutable

Los hechos publicados en el grafo no pueden modificarse. Esta no es una restricción semántica sino una consecuencia criptográfica directa: cada credencial está cubierta por una firma ECDSA. Modificar cualquier campo de un hecho ya publicado invalida la firma que lo cubre, haciendo la alteración detectable por cualquier verificador. El pasado del grafo es, por construcción, inalterable.

Cuando un hecho contiene un error, la corrección no borra el original — añade un nuevo hecho que lo complementa o corrige, firmado por quien tiene autoridad para hacerlo. El grafo mantiene la historia completa: el error original, la corrección, quién la emitió y cuándo. Cualquier verificador puede reconstruir exactamente qué era válido en cualquier momento del pasado.

Esta propiedad distingue a Trusteando de los sistemas de registro tradicionales, donde modificar un campo es operativamente posible y a menudo indetectable. En Trusteando, cualquier intento de alterar el pasado es criptográficamente visible. La confianza en el sistema no depende de que nadie quiera manipular los registros — depende de que la manipulación es imposible sin dejar evidencia.

La convención since/until y el modelo plan/execution de A.24 son la expresión temporal de este principio: since/ y until/ registran cundo algo ocurrió, plan/ declara la intención, y execution/ acumula los hechos reales según van ocurriendo. El plan puede cambiar; los hechos de ejecución ya registrados no.

Cualquier hecho publicado debe tratarse como inalterable. Modificarlo rompe la firma que lo cubre y destruye la cadena de confianza que depende de él.

2.11 Publicidad por defecto y privacidad declarada

Todo lo que existe en el grafo es público salvo lo que esté bajo private/. La privacidad se declara explícitamente; la publicidad es el estado natural. Esta inversión del modelo tradicional — donde hay que declarar qué se comparte — hace que el grafo sea auditable por defecto y que la privacidad sea una elección consciente, no una consecuencia del silencio.

La carpeta private/ resuelve tres niveles de privacidad. Primero, contenido privado con relación pública: el nodo es visible pero sus propiedades están ocultas. Segundo, relación privada con existencia de privacidad pública: la carpeta private/ es visible pero no su contenido — un verificador sabe que algo existe ahí sin saber qué. Tercero, privacidad total bajo carpeta opaca: ni siquiera el tipo de relación es visible. En los tres casos, acceder al contenido requiere elevar una consulta al agente que controla la carpeta, que decide si concede acceso mediante grantReveal().

Aplicado a una universidad: universidad.es/personal/profesores/private/ oculta la identidad de determinados profesores. Un verificador externo ve que existen entradas ocultas bajo esa carpeta, pero no quiénes son ni cuántos. Para acceder debe elevar una consulta al agente profesores/, que decide si concede el acceso. Esto resuelve el problema de la privacidad del grafo de relaciones descrito en la sección 12.7: la existencia de la arista puede ocultarse, no solo su contenido.

Los nodos también pueden modelarse como objetos con métodos y valores, siguiendo la analogía de la programación orientada a objetos. La carpeta es el objeto, las subcarpetas son métodos o colecciones, y los corchetes son propiedades o valores — inmutables una vez firmados. Un acuerdo entre partes ilustra esta estructura:

```
acuerdo-xyz/                          ← object
├── [state trusteando]/               ← object.state
├── participantes/                    ← object.collection
│   ├── [empresaA proveedor]/          ← object.property (inmutable)
│   └── [empresaB cliente]/            ← object.property (inmutable)
├── plan/                             ← object.method (intención)
│   ├── [objetivo "servicios 2026"]/
│   └── [state approved]/
├── execution/                        ← object.method (realidad)
│   └── firma/since/2026-03-18/        ← hecho (ya es historia,
inmutable)
└── private/             ← privado, requiere grantReveal()
```

La analogía con programación orientada a objetos es ilustrativa pero incompleta: a diferencia de una clase con estado privado, aquí no hay encapsulación por defecto. Todo es público salvo lo que se oculta explícitamente. Es más cercano a un record inmutable en un lenguaje funcional: las propiedades firmadas no se modifican, las correcciones añaden nuevos registros, y el historial completo es siempre accesible.

2.12 La jerarquía de carpetas es la jerarquía de claves

El protocolo tiene una propiedad estructural que conecta su organización visible con su fundamento criptográfico: la jerarquía de carpetas y la jerarquía de claves son isomorfas. Cada carpeta es un nodo, y cada nodo tiene una clave derivada directamente de su posición en el árbol. Recorrer el árbol de carpetas es verificar la cadena criptográfica. No hay infraestructura de clave pública separada — la estructura de carpetas es la infraestructura criptográfica.

Esta isomorfía tiene una consecuencia práctica importante: cualquier entidad que controle una carpeta controla automáticamente la clave asociada a ella, y puede delegar claves a subcarpetas mediante grant_key. La autoridad sobre una rama del árbol es a la vez autoridad

sobre la información publicada en esa rama y autoridad sobre las claves criptográficas que la protegen. No hay posibilidad de que estas dos autoridades divergan. La sección 4.11 formaliza este principio con la clase TrusteandoNode.

3. Estructura de una credencial

Una credencial Trusteando tiene tres campos:

Campo	Descripción
public_key	Clave pública del firmante. Identifica al emisor de la credencial de forma verificable.
hash(mensaje)	Hash SHA-256 del contenido que se afirma. El contenido puede mantenerse privado — solo el hash es necesario para la verificación.
hash_verificacion	HMAC-SHA-256 del hash(mensaje) usando la clave compartida entre el firmante y su nivel superior. Solo el nivel superior puede verificarlo — confirma que la credencial pertenece a alguien reconocido por ese nivel.
auth_signature	Firma ECDSA del msg_hash con la clave privada del firmante. Solo el poseedor de esa clave privada puede generarla. Cualquier tercero puede verificarla con la clave pública del firmante. El root guarda un compromiso firmado en el alta que vincula esta clave con la identidad real del firmante, permitiendo resolución de disputas con respaldo legal.

La credencial no es un documento — es la presencia de un nodo en una carpeta del espacio de identidad de su superior. La estructura de carpetas es la credencial. Por ejemplo:

```
universidad.es/profesores/juan-escobar          ← Juan es Profesor
universidad.es/doctores/juan-escobar            ← Juan es Doctor
universidad.es/profesores/since/2021-09-01/juan-escobar   ←
incorporación
universidad.es/profesores/until/2024-06-30/juan-escobar   ← baja
```

La dimensión temporal está codificada en la propia estructura de carpetas — bajo control de la universidad, no de Juan. Juan no puede alterar las fechas de incorporación ni de baja. La revocación de una credencial específica es simplemente eliminar al nodo de la carpeta correspondiente, sin afectar a las demás.

Adicionalmente, cada credencial puede incluir un bloque de metadatos firmados:

```
{
  "issuer":         "SHA-256(url_del_emisor)",
  "subject":        "SHA-256(url_o_identificador_del_sujeto)",
  "public_key":     "base64(clave_publica_del_emisor)",
  "msg_hash":       "SHA-256(contenido_de_la_credencial)",
  "hmac":           "HMAC-SHA-256(msg_hash, clave_compartida)",
  "auth_signature": "ECDSA.sign(msg_hash, clave_privada_firmante)",
  "timestamp":      1710000000
}
```

El mensaje cuyo hash se incluye puede tener cualquier estructura. El protocolo no impone un esquema. Ejemplos de mensajes válidos:

```
{ "role": "Profesor Asociado", "department": "Informatica" }
{ "fact": "email_sent", "from": "a@uni.es", "to": "b@uni.es", "date":
"2026-03-17" }
{ "claim": "age_over_18", "verified_by": "SHA-256(universidad.es)" }
{ "presence": "SHA-256(https://venue.es)", "timestamp": 1710000000 }
```

4. La cadena de confianza

4.1 Estructura general

El sistema organiza la confianza en niveles. Cada nivel puede emitir credenciales sobre los niveles que reconoce. La cadena tiene la siguiente forma general:

```
Trusteando root
  └─ reconoce a universidad.es
       └─ reconoce a juan@universidad.es (Profesor Asociado)
            └─ juan firma credenciales verificables

Trusteando root
  └─ reconoce a empresa.com
       └─ reconoce a correo.empresa.com (servidor de correo)
            └─ correo certifica comunicaciones entre empleados
```

El número de niveles no está limitado por el protocolo. Cada entidad decide con qué profundidad organiza su espacio de identidad interno.

4.2 Establecimiento de la clave compartida

Cada par de niveles adyacentes comparte una clave simétrica establecida mediante intercambio Diffie-Hellman sobre curva elíptica (ECDH). Este es el mismo mecanismo que usa TLS en cada conexión HTTPS — probado, estandarizado, y bien comprendido.

El proceso de establecimiento ocurre una sola vez por par:

```
# La entidad inferior genera su par de claves efímero
privada_inferior, publica_inferior = ECDH.generate_keypair(curve=P-
256)

# El nivel superior genera su par efímero para este intercambio
privada_superior, publica_superior = ECDH.generate_keypair(curve=P-
256)

# Intercambian claves públicas (canal autenticado)

# Cada parte deriva la misma clave compartida independientemente
clave_compartida = ECDH.derive(privada_inferior, publica_superior)
clave_compartida = ECDH.derive(privada_superior, publica_inferior)
# → mismo resultado en ambos lados

# La clave compartida se usa para generar y verificar HMAC
hash_verificacion = HMAC-SHA-256(msg_hash, clave_compartida)
```

La propiedad fundamental del intercambio Diffie-Hellman es que ninguna de las dos partes conoce la clave privada de la otra, y nadie que observe el intercambio de claves públicas puede derivar la clave compartida. El nivel superior no puede impersonar al inferior porque no tiene su clave privada.

El root no conoce la clave compartida entre la universidad y Juan. La universidad no conoce la clave compartida entre el root y otras entidades. Cada par tiene una clave que es exclusivamente suya.

4.3 Verificación en cadena

Para verificar que una credencial firmada por Juan como Profesor Asociado es válida, el verificador sigue estos pasos:

```
1. Obtener la credencial de Juan:
   (public_key_juan, msg_hash, hmac, auth_signature)

2. Consultar universidad.es/trusteando — ¿está public_key_juan
registrada?
   → sí: Juan pertenece al espacio de identidad de la universidad

3. Verificar AUTENTICIDAD — pedir a la universidad que verifique hmac:
   universidad comprueba: HMAC(msg_hash, clave_compartida_juan_uni) ==
hmac
   → la credencial viene de alguien reconocido por la universidad

4. Verificar AUTORÍA — con la clave pública de Juan:
   ECDSA.verify(msg_hash, auth_signature, public_key_juan) == válido
   → solo Juan pudo generar esta firma

5. ¿Es universidad.es un agente reconocido?
   → consultar registro público del root
   → root firmó: SHA-256(universidad.es) → public_key_universidad
   → válido

6. Credencial verificada — autenticidad y autoría confirmadas.
```

El verificador no necesita conocer la clave compartida entre Juan y la universidad — solo necesita que la universidad confirme la validez del HMAC. La universidad no revela la clave compartida: solo responde sí o no a la consulta de verificación.

Nota sobre escalabilidad. El paso 3 requiere que el nodo superior — la universidad en el ejemplo — tenga un endpoint de verificación disponible para cada consulta. En organizaciones con muchos miembros y alto volumen de verificaciones esto puede convertirse en un cuello de botella. Hay dos mecanismos complementarios para mitigarlo.

El primero es la caché con firma. El nodo superior publica periódicamente un documento firmado que lista los HMACs válidos para un período dado, con su TTL correspondiente. El verificador descarga ese documento una vez y verifica localmente durante toda la vigencia del TTL, sin consultar al superior en cada verificación individual. Este mecanismo ya está contemplado en la convención cache/ del protocolo.

El segundo es la verificación por lotes mediante árbol de Merkle. El nodo superior publica periódicamente la raíz de un árbol de Merkle construido sobre todos los HMACs válidos del período. El nodo miembro puede entonces demostrar que su HMAC está incluido en ese árbol presentando únicamente su rama de prueba — un conjunto logarítmico de hashes que el

verificador puede comprobar localmente sin necesidad de consulta en línea al superior y sin que el superior revele el conjunto completo de HMACs. Ambos mecanismos son compatibles entre sí y con el flujo de verificación descrito en los pasos anteriores.

Un tercer mecanismo, complementario a los anteriores, es el modelo de réplicas observadoras con consenso por suma. N nodos independientes — entidades reputadas con identidad verificable en el propio grafo: universidades, organismos públicos, colegios profesionales — mantienen copias de los espacios de identidad que observan. El verificador consulta K de esas réplicas y obtiene votos binarios: 1 si la credencial es válida según esa réplica, 0 si no. La suma de votos determina el resultado.

El emisor publica su estructura de carpetas sin necesitar saber nada del consenso. Las réplicas son observadores pasivos que sincronizan mediante diffs criptográficos — deltas mínimos que describen solo lo que ha cambiado — en lugar de replicar el árbol completo. Esto hace la propagación eficiente incluso con conexiones modestas, y los diffs funcionan como registro de cambios nativo: guardarlos equivale a tener el historial completo de cómo ha evolucionado la confianza de una entidad.

La propagación es escalonada: las réplicas más rápidas reciben el diff primero y pueden responder consultas casi de inmediato, mientras las lentas se actualizan en segundo plano. Para evitar ambigüedad durante la ventana de inconsistencia el protocolo define un umbral de histéresis: solo se acepta como válido un resultado que supere el umbral configurado (por ejemplo, 80 de 100 réplicas coinciden). Un resultado por debajo del umbral es estado indefinido — no verdad parcial — que el verificador trata como no concluyente y reintenta tras un intervalo. Las réplicas permanentemente desactualizadas pueden solicitar sincronización completa a las réplicas rápidas mediante pull-sync.

La analogía es una CDN de identidad: las réplicas distribuyen certidumbre igual que una red de distribución de contenidos distribuye datos. La carga computacional de cada consulta es mínima — una operación aritmética sobre hashes — lo que hace el sistema accesible a dispositivos con recursos limitados. Añadir réplicas aumenta la resiliencia sin penalizar la velocidad de consulta individual. El coste de una consulta es equivalente a una petición HTTP estándar.

El riesgo principal es el ataque Sybil: si un atacante controla la mayoría de las réplicas puede validar relaciones inexistentes. La mitigación es estructural: las réplicas no son nodos anónimos sino entidades con identidad verificable en el propio grafo. Observadores con intereses contrapuestos — universidades de países distintos, organismos de diferentes jurisdicciones — hacen que la colusión sea socialmente improbable además de criptográficamente costosa. La especificación formal de este mecanismo se abordará en versiones futuras. La operación de verificación de cada réplica es directamente expresable en términos de la clase TrusteandoNode definida en la sección 4.11: verify_child_authorship encapsula exactamente el cálculo que cada réplica ejecuta para validar una credencial.

4.4 Separación entre autenticidad y autoría

La introducción de auth_signature resuelve un problema estructural importante: sin ella, el nivel superior podría suplantar al inferior. Conoce la clave compartida, puede generar un HMAC válido, y podría construir una credencial aparentemente legítima atribuida a Juan. Con auth_signature eso es imposible — la clave privada de Juan nunca sale de su dispositivo.

Las dos propiedades son verificables independientemente:

- Autenticidad — verificada por el nivel superior mediante el HMAC. Responde a: ¿viene esta credencial de alguien que yo reconozco?
- Autoría — verificada por cualquier tercero con la clave pública del firmante mediante ECDSA. Responde a: ¿fue específicamente esta persona y no otra?

Esta separación hace la cadena de confianza robusta frente a compromisos internos: incluso si un nivel superior es malicioso o está comprometido, no puede fabricar credenciales atribuibles a sus agentes inferiores. La sección 4.11 formaliza este mecanismo en términos del modelo TrusteandoNode: respond_to_challenge garantiza la autoría porque solo el titular de la clave puede producir la respuesta correcta, y verify_child_authorship permite al padre verificar sin que el hijo participe.

4.5 Non-repudiation y resolución de disputas

La firma ECDSA establece la evidencia técnica de autoría. Sin embargo el non-repudiation completo — la imposibilidad de negar la autoría incluso cuando ninguna de las partes quiere asumir responsabilidad — requiere algo más que criptografía.

En el momento del alta, el firmante suscribe ante el root un compromiso de responsabilidad: acepta que la clave privada registrada es suya y que asume responsabilidad por las credenciales firmadas con ella. El root custodia ese compromiso.

En caso de disputa en que ninguna de las partes quiere asumir la autoría:

```
1. El root presenta el compromiso firmado por Juan en el alta
2. La evidencia criptográfica vincula ese compromiso con la credencial
disputada
   mediante auth_signature verificable con la clave pública registrada
3. La responsabilidad queda establecida aunque Juan no quiera
reconocerla
```

Este mecanismo reconoce explícitamente que el non-repudiation no es solo un problema criptográfico. La criptografía establece la evidencia — el marco legal le da fuerza vinculante. Es el mismo modelo que usan los sistemas de firma electrónica cualificada bajo el reglamento eIDAS en la Unión Europea. El protocolo establece la evidencia; el marco institucional le da fuerza. Ambas capas son necesarias y ninguna sustituye a la otra.

4.6 El rol del nodo raíz

El root tiene tres responsabilidades:

- Una vez por agente: establecer la clave compartida con la entidad mediante ECDH, y publicar en su registro la asociación entre el entity_id de la entidad y su clave pública.
- Mantener un registro público firmado de agentes reconocidos, actualizado periódicamente con un TTL que permite a los verificadores cachearlo localmente.
- Custodiar las claves de emergencia registradas y aprobar migraciones de identidad cuando se le presenta evidencia válida.

El registro de claves de emergencia con el root es asíncrono — un nodo puede publicar su hash_publico en su propia web y operar con normalidad antes de que el root lo registre. El root procesa los registros según su capacidad. El sistema no se bloquea mientras tanto.

4.7 Resiliencia de identidad — la clave de emergencia

Cada nodo genera en el momento de su creación una clave de emergencia que nunca comparte con su superior ni con ningún otro nodo. Con ella calcula su hash_publico:

```
clave_emergencia  ← generada localmente, nunca compartida con el
superior
hash_publico = hash(entity_id, clave_emergencia)
```

El hash_publico se publica en la propia web del nodo como parte de su identidad pública, en una ruta estándar que cualquier implementación del protocolo sabe dónde buscar:

```
universidad.es/profesores/juan-escobar/hash_publico/a3f9e2b1c4d7...
```

Cualquier tercero puede ver este valor. Solo Juan puede demostrar que lo generó, porque solo él conoce la clave_emergencia que lo produce. El hash_publico está anclado en dos lugares independientes: la propia web del nodo y, cuando el root lo registra, el registro central. Si cualquiera de los dos falla, el otro sostiene la cadena.

Migración voluntaria a un nuevo superior

Cuando Juan quiere migrar a una nueva universidad, el proceso es:

```
1. Juan revela su clave_emergencia a universidad-b.es

2. Universidad-b verifica contra el hash_publico publicado por Juan:
   hash(entity_id_juan, clave_emergencia) == hash_publico
   → la adopción es legítima y voluntaria — Juan entregó la clave

3. Universidad-b comunica al root que conoce la clave_emergencia de
Juan.
   Root verifica: hash(entity_id_juan, clave_emergencia) ==
hash_publico
   → ecuación válida: migración confirmada. No hay discrecionalidad.

4. Universidad-b crea la entrada de Juan en su espacio:
   universidad-b.es/profesores/juan-escobar/

5. Universidad-b registra el vínculo en su propia tabla
old_identities,
   bajo su control — no bajo el control de Juan:
   universidad-b.es/old_identities/
     hash_nuevo → [ hash_nuevo, hash_antiguo_1, hash_antiguo_2, ... ]

   La cadena completa de identidades anteriores se publica entera.
   Universidad-b la recibió de universidad-a en el proceso de
adopción,
   añade la nueva entrada, y la firma con su propia clave.

6. Juan genera una nueva clave_emergencia y publica nuevo
hash_publico.
   La clave anterior queda invalidada.
```

La tabla old_identities es un registro institucional de la universidad, no de Juan. Juan no puede modificarla ni omitir entradas. Está firmada por la universidad con la misma autoridad que cualquier otra credencial que emite.

El rol del root en este proceso es puramente algorítmico: ejecuta la función pública `hash(entity_id, clave_emergencia) == hash_publico` y devuelve verdadero o falso. No tiene discrecionalidad para denegar una migración cuya ecuación se cumple, ni para aprobar una cuya ecuación no se cumple. Cualquier nodo que ejecute la misma función sobre los mismos datos obtendría idéntico resultado. El root no custodia ningún estado privilegiado — solo ejecuta un algoritmo público.

Publicar la cadena completa — no solo el vínculo inmediato anterior — permite a cualquier agente del sistema verificar el historial completo de Juan con una sola consulta, sin tener que saltar de servidor en servidor siguiendo eslabones uno a uno. Cada migración extiende la cadena por un elemento; la universidad receptora la hereda, la extiende, y la publica entera firmada.

Adopción temporal por el root cuando el superior desaparece

Si el superior de Juan desaparece — la universidad cierra, pierde el dominio, o es revocada — Juan puede identificarse ante el root con su clave_emergencia:

```
1. Juan presenta al root su clave_emergencia
2. Root verifica: hash(entity_id_juan, clave_emergencia) ==
hash_publico registrado
3. Root acoge a Juan como nodo huérfano bajo su tutela directa
4. Juan opera con normalidad mientras encuentra un nuevo superior
5. Cuando un nuevo superior lo adopta, sigue el proceso de migración
estándar
```

Esta propiedad garantiza que la identidad de un nodo sobrevive a la desaparición de su superior. No empieza de cero — continúa con su historial, sus credenciales anteriores verificables, y su reputación acumulada. Es análogo a la portabilidad del número de teléfono, pero para identidad criptográfica: el titular controla la portabilidad, no el operador.

Degradación con gracia

Si el root está saturado o no disponible, los nodos existentes siguen operando con normalidad. Las verificaciones no requieren al root — consultan directamente las webs de las entidades. Solo se bloquean los registros nuevos y las migraciones, no el uso cotidiano del sistema.

4.9 Raíces de confianza distribuidas

Con el tiempo, las entidades con reconocimiento público suficiente se convierten en raíces de confianza naturales. Un verificador que conoce y confía en una universidad puede verificar credenciales de sus profesores sin consultar al root de Trusteando.

El sistema evoluciona de jerárquico a policéntrico de forma orgánica. El root de Trusteando hace bien su trabajo cuando deja de ser necesario para el uso cotidiano — cuando la red de confianza tiene suficientes nodos con reputación propia para sostenerse sin una autoridad central única.

4.10 Raíces múltiples e independencia de la identidad

Esta sección extiende el modelo criptográfico del protocolo para incorporar redundancia de testigos y desacoplamiento de la identidad de la infraestructura web. No reemplaza el modelo base — lo complementa. Una entidad puede tener identidad derivada de URL, identidad autónoma con secretos propios, o ambas vinculadas mediante old_identities/.

4.10.1 Independencia del identificador

En el modelo base el entity_id se deriva de la URL canónica. Esta decisión pragmática ancla la identidad en una presencia web preexistente pero crea dependencia de la infraestructura DNS y de la continuidad del dominio. Para entidades que requieren máxima resiliencia, el protocolo permite una modalidad alternativa: identidad autónoma. La entidad genera localmente su identificador y un secreto que solo ella conoce:

```
object.id          = hash("universidad-de-málaga")
object.secret_base = hash("frase secreta conocida solo por la
universidad")
```

La URL pasa a ser el ancla de publicación — donde la entidad publica sus datos — pero no el ancla de identidad. La identidad puede existir antes de que haya URL, puede sobrevivir a la pérdida del dominio y al bloqueo DNS, y puede mantenerse aunque el root original desaparezca.

4.10.2 Múltiples secretos para múltiples contextos

Una entidad puede generar diferentes secretos para diferentes propósitos, manteniendo su aislamiento:

```
secret_identidad  = hash("frase para identidad principal")
secret_academico  = hash("frase para credenciales académicas")
secret_emergencia = hash("frase para migraciones y rescate")
```

Comprometer un secreto no afecta a los demás. Cada secreto genera su propia cadena de derivación criptográfica. Una misma entidad puede tener una identidad civil reconocida por roots gubernamentales, una identidad profesional reconocida por roots colegiales, y una identidad operativa reconocida por roots técnicos, sin que el compromiso de una afecte a las otras.

4.10.3 Red de raíces múltiples

Dado que el root es un algoritmo público trivial, cualquier nodo con suficiente reputación puede actuar como root. No hay distinción fundamental entre un root y un nodo de alta reputación: ambos ejecutan el mismo algoritmo público. Esto permite un ecosistema con miles de roots potenciales. La entidad elige al azar un conjunto de N roots (por ejemplo 5) y envía su solicitud a cada uno. Para cada root i y cada secreto j se deriva una clave pública específica:

```
public_key[i][j] = hash(object.id + secret_j + root[i].secret)
```

El resultado es una matriz de verificación de dimensiones roots × secretos. Cada celda es una prueba independiente de que la entidad conoce el secreto j y ha sido reconocida por el root i. No hay ningún root obligatorio. La diversidad geográfica, jurísdica y organizativa de los roots elegidos determina la resiliencia: un atacante que controla una jurisdicción o compromete una organización solo afecta a los roots de ese ámbito.

4.10.4 Quórum de validación

Para operaciones sensibles — emisión de credenciales de alto valor, migración de identidad, resolución de disputas — la entidad puede requerir un quórum de K roots (con K ≤ N). Una operación es válida si presenta firmas válidas de al menos K roots independientes. Usando la gramática del protocolo:

```
operacion/[quorum 3]/
    [testigo root-23]/
    [testigo root-45]/
    [testigo root-78]/
```

Un verificador que encuentre el marcador [quorum 3] sabe que debe comprobar al menos 3 firmas de roots distintos antes de aceptar la operación. El mismo mecanismo aplica a nodos certificadores intermedios replicados en múltiples instancias: alterar un campo crítico — como la fecha de emisión de un título — requiere el consenso del quórum de instancias, no el control de una sola.

4.10.5 Propiedades resultantes

Resiliencia ante compromiso local. Un atacante que comprometa un root puede falsear las claves de ese root pero no las de los otros. Si el quórum requerido es 3 y hay 5 roots, el atacante necesita comprometer 3 roots independientes, cada uno con su propia seguridad, gobernanza y localización.

Independencia de infraestructura. La identidad sobrevive a cambio de dominio, pérdida del servidor web, bloqueo DNS y desaparición del root original.

Escalabilidad de la confianza. Cualquier nodo con reputación puede ofrecerse como root. No hay límite superior. Las entidades pueden elegir sus roots al azar, por geografía, por ámbito institucional, o por cualquier criterio.

Separación de contextos. Los diferentes secretos permiten que una misma entidad tenga identidades en distintos contextos institucionales sin que el compromiso de una afecte a las otras.

Esta extensión convierte Trusteando en un sistema policefálico por diseño desde el momento del alta. No hay un único punto de fallo, no hay una única autoridad de la que dependa la existencia de una identidad. La identidad es del sujeto, respaldada por un quórum de testigos independientes que ejecutan un algoritmo público. El resultado es un sistema que hereda la robustez de los sistemas de consenso distribuido sin necesidad de blockchain, porque la prueba de existencia es la multiplicidad de firmas independientes, no una cadena inmutable compartida.

4.11 El núcleo del protocolo: cuatro funciones

Trusteando se construye sobre una premisa tan simple como profunda: cada carpeta del grafo es un nodo, y cada nodo tiene una clave privada que se deriva directamente de su ruta en la jerarquía. No hay infraestructura de clave pública separada. No hay certificados. No hay autoridades externas. La estructura de carpetas es la infraestructura criptográfica. Todo el protocolo se reduce a cuatro funciones que caben en unas pocas líneas de código. Aunque parecen simples, resuelven todos los problemas planteados sin casos especiales ni cabos sueltos.

```
class TrusteandoNode:
    def __init__(self, key):
        self.key = key

    @staticmethod
    def reduce_hash(seed, elements):
        """Operación primitiva: fold hash por la izquierda"""
        result = seed
        for element in elements:
            result = hash(result + element)
        return result

    def grant_key(self, child_path_segment):
        """El padre otorga una clave al hijo"""
        return hash(self.key + child_path_segment)

    def respond_to_challenge(self, context_elements):
        """El hijo responde a un reto sin revelar su clave"""
        return self.reduce_hash(self.key, context_elements)

    def verify_child_authorship(self, child_path_segment,
context_elements, proof):
        """Solo el padre puede verificar la autoría de un hijo"""
```

```
        child_key = self.grant_key(child_path_segment)
        return self.reduce_hash(child_key, context_elements) == proof
```

4.11.1 reduce_hash: la operación primitiva

reduce_hash es la única operación criptográfica que necesita el protocolo. Toma una semilla y una lista de elementos y devuelve un hash que depende de todos ellos en orden. Es determinista (mismos inputs producen mismo output), unidireccional (el output no permite recuperar ningún input), sensible al orden (reduce_hash(s, [a,b]) es distinto de reduce_hash(s, [b,a])), y composable (se puede verificar por etapas).

4.11.2 grant_key: el padre otorga claves al hijo

Cuando un nodo padre incorpora un hijo con un segmento de ruta, le otorga una clave derivada: child_key = hash(parent_key + child_path_segment). El padre entrega child_key al hijo por un canal seguro. A partir de ese momento el hijo conoce su clave, el padre puede recalcularla cuando sea necesario, y nadie más la conoce. El hijo nunca conoce la clave del padre — la dirección de confianza es estrictamente descendente.

4.11.3 respond_to_challenge: el hijo responde sin revelar su clave

Cuando un nodo quiere probar su identidad para un contexto específico, genera una prueba: prueba = reduce_hash(self.key, [verificador_id, hash(contenido), timestamp]). El nodo no revela su clave, solo demuestra que la conoce. La prueba es específica para ese verificador, ese contenido y ese momento — no puede reutilizarse ni transferirse.

Ejemplo: un profesor publica un comentario en un foro externo con su profesor_id. El moderador quiere verificar que es realmente ese profesor. El moderador envía al profesor: [moderador_id, hash(comentario), timestamp]. El profesor calcula respond_to_challenge([moderador_id, hash(comentario), timestamp]) y envía la respuesta. El moderador la envía al nodo padre — el que aparece en la ruta del profesor_id — junto con el contexto. El padre verifica.

4.11.4 verify_child_authorship: solo el padre verifica

El verificador externo no elige a quién preguntar arbitrariamente — pregunta siempre al nodo padre del hijo que emitió la prueba. Si el identificador es uma.es/profesores/juan, la solicitud de verificación va a uma.es/profesores. El padre recalcula la clave del hijo con grant_key y verifica la prueba con reduce_hash. Si coinciden, la prueba es auténtica. El hijo no participa en la verificación.

4.11.5 Propiedades de seguridad que emergen de la simplicidad

No repudio: solo quien tiene la clave puede producir respond_to_challenge correcto. No reutilizable: context_elements incluye verificador_id y timestamp, únicos por interacción. No transferible: la respuesta solo es válida para ese conjunto específico de elementos. Sin revelación de clave: reduce_hash es unidireccional; la respuesta no permite recuperar la clave. Verificación por el padre: verify_child_authorship recalcula sin necesitar al hijo. Jerarquía de confianza: grant_key es la única forma de crear un nodo hijo válido, siempre de padre a hijo. Verificación externa: cualquier sistema — foros, correo, documentos — puede usar el mismo mecanismo enviando un reto y verificando la respuesta con el padre. No hay casos especiales. No hay estado oculto. Todo el sistema es determinínista, auditable y verificable por construcción.

5. Dos niveles de existencia

Una entidad puede operar en dos niveles distintos en el sistema:

Nivel	Descripción
Identidad declarada	La entidad publica sus hashes y claves en su propia URL. La cadena es coherente y verificable por cualquiera que conozca la entidad. No hay garantía externa para verificadores que no la conocen. Útil para sistemas internos, comunidades cerradas, o para construir la cadena antes de solicitar reconocimiento formal.
Identidad reconocida	El root ha establecido una clave compartida con la entidad y publicado su certificado de agente. La cadena es verificable por cualquier tercero sin conocimiento previo de la entidad. La confianza es transitiva hacia los niveles inferiores.

Una entidad puede operar en nivel 1 indefinidamente. El nivel 2 añade verificabilidad para terceros desconocidos — no es un requisito para que el sistema funcione.

6. Privacidad y divulgación selectiva

6.1 Privacidad por defecto

El verificador ve solo el resultado de la verificación: la credencial es válida o no lo es. La cadena completa de confianza — qué entidades están implicadas, qué dice el mensaje original — queda oculta salvo que el titular de la credencial decida revelarla explícitamente.

El hash del mensaje permite verificar la integridad sin revelar el contenido. El emisor puede demostrar que el mensaje no ha sido alterado sin mostrar el mensaje mismo.

6.2 Divulgación selectiva

El titular de una credencial puede elegir qué revelar en cada contexto:

- Solo el resultado: la credencial es válida — sin revelar quién la emitió ni qué dice
- El emisor pero no el contenido: esta credencial viene de universidad.es — sin revelar el rol específico
- La cadena completa: universidad.es certifica que Juan Escobar es Profesor Asociado del Departamento de Informática

Esta propiedad permite casos de uso como verificación de mayoría de edad sin revelar la fecha de nacimiento, o verificación de pertenencia a una organización sin revelar el cargo.

6.3 Lo que el sistema no oculta

El hecho de que una entidad está registrada como agente en el root es público — forma parte del registro de agentes reconocidos. El entity_id (hash de la URL) de una entidad es derivable por cualquiera que conozca su URL. El protocolo no está diseñado para ocultar la existencia de las entidades participantes, solo para proteger el contenido de las credenciales que emiten.

6.4 Privacidad dinámica mediante gestor

La carpeta private/ es un mecanismo estático: la información existe en el servidor pero no se expone por defecto. Para casos que requieren control de acceso dinámico — donde la decisión de revelar depende del contexto de cada solicitud — el protocolo permite un modelo de gestor. El gestor es software que recibe la petición, identifica quién pregunta, decide si concede acceso, y sirve la información solo en ese momento bajo demanda. En Trusteando este gestor es el agente que controla la carpeta private/, implementado mediante requestReveal() y grantReveal(). La URL estática es el modelo para información pública; el gestor es el modelo para información privada bajo demanda. Los dos modelos son complementarios y pueden coexistir en el mismo nodo.

7. Casos de uso

El protocolo es agnóstico al contenido de las credenciales. Los casos de uso siguientes ilustran la generalidad del mecanismo — no son los únicos posibles.

7.1 Credenciales académicas y profesionales

Una universidad registra a sus profesores en su espacio de identidad. Un colegio de médicos registra a sus colegiados. Un bar de abogados registra a sus miembros. Cualquier verificador puede comprobar la credencial sin llamar a ningún servidor central — consultando la URL de la entidad emisora y verificando la firma.

7.2 Reseñas verificadas por presencia

Un establecimiento registra tokens de presencia temporales — por ejemplo mediante un código QR rotativo disponible solo en el local. Un usuario que estuvo físicamente en el local puede obtener un token de presencia y usarlo para firmar una reseña. La reseña lleva una credencial de presencia que cualquiera puede verificar: esta persona estuvo en este lugar.

> *Este es el caso de uso que motivó el desarrollo inicial del protocolo. La verificación de presencia es un caso particular de credencial verificable — no un mecanismo distinto.*

7.3 Verificación de comunicaciones

El servidor de correo de una empresa, registrado como agente delegado por la empresa, puede emitir credenciales que certifiquen que un correo fue enviado entre dos partes en una fecha determinada. El contenido del correo puede permanecer privado — solo el hash del mensaje y la credencial de envío son necesarios para la verificación.

7.4 Verificación de condición sin revelar datos

Una entidad con autoridad sobre datos personales — un registro civil, una universidad, un sistema de salud — puede emitir credenciales de condición: mayor de edad, titulado universitario, profesional sanitario colegiado. El verificador recibe la confirmación de la condición sin acceder a los datos subyacentes.

7.5 Delegación de autoridad dentro de organizaciones

Una empresa puede registrar como agentes delegados sus divisiones, departamentos, o sistemas internos. Cada uno puede emitir credenciales en su ámbito. La cadena de delegación es verificable externamente sin exponer la estructura interna de la organización.

7.6 Autoría verificable de contenido

Un autor puede firmar un documento, un artículo, o cualquier contenido digital con su credencial Trusteando. Cualquier verificador puede comprobar que el contenido fue producido por esa persona y que no ha sido alterado desde entonces, sin necesidad de terceros de confianza adicionales.

7.7 Coordinación de emergencias multi-agencia

La estructura jerárquica de Trusteando no solo sirve para identidad estática — es un modelo natural para representar estructuras de mando y coordinación en tiempo real. En incidentes donde intervienen múltiples organismos — policía, bomberos, servicios sanitarios, protección civil — la capacidad de publicar y leer estados operativos en carpetas estructuradas elimina la fricción de la comunicación por radio y la pérdida de información en los relevos.

Cada interviniente es un nodo. Cada equipo publica su ubicación, estado y necesidades bajo su propia URL. El mando del incidente publica órdenes usando la convención plan/execution: plan/ contiene el objetivo y el plazo; execution/ lo actualizan los equipos con su progreso real. La ausencia de execution/end/ indica que la orden sigue activa. Las solicitudes de coordinación entre agencias — por ejemplo, policía solicitando a bomberos que despejen un acceso — son entradas de carpeta que ambas partes pueden ver, aceptar y ejecutar con trazabilidad completa.

```
emergencias.madrid.es/incidents/2026-03-18/incendio-xxx/
├── command/
│   ├── orders/orden-001/
│   │   ├── plan/      ← objetivo, plazo, equipos asignados  (firma: mando)
│   │   └── execution/ ← partes, incidencias, end/ cuando completa  (firma:
equipo)
├── firefighters/
│   ├── deployment/equipos/      ← quién está dónde
│   └── requests/police/     ← coordinación entre agencias
├── police/
└── medics/
```

El resultado es trazabilidad completa: cada acción, cada orden, cada cambio de estado queda firmado y fechado. El mando ve en una sola pantalla el estado real de todos los equipos sin una llamada de radio. En un relevo, el equipo entrante consulta execution/ del equipo saliente y retoma sin pérdida de información. Tras el incidente, el registro completo es verificable y sirve como base para la reconstrucción forense.

Una ventaja adicional es la velocidad de diseño organizativo. Ante una emergencia nueva — un tipo de incidente poco frecuente, una zona geográfica desconocida, una combinación inusual de agencias — un modelo de lenguaje puede generar en segundos una propuesta completa de estructura de carpetas adaptada al contexto: qué nodos intervienen, qué carpetas necesita cada uno, cómo se expresan las relaciones de mando y las solicitudes entre agencias. El equipo humano evalúa la propuesta, la ajusta si es necesario, y la estructura está operativa de inmediato. Lo que antes requería reuniones y documentos de procedimiento puede resolverse en minutos, con el protocolo como lenguaje común que tanto humanos como sistemas automáticos entienden sin ambigüedad.

7.8 Interoperabilidad entre administraciones y organismos

La interoperabilidad entre administraciones públicas es hoy un problema sin resolver estructuralmente. Cada par de organismos que necesita intercambiar datos requiere un convenio bilateral, un formato de intercambio específico, y una pasarela técnica que traduzca entre sistemas heterogéneos. El coste de integración crece con el número de actores.

Trusteando invierte esta dinámica gracias a la naturaleza del protocolo: cualquier sistema — una base de datos relacional, una API existente, un registro heredado — puede exponer sus datos a través de una interfaz que respeta el esquema de carpetas sin necesidad de migrar ni reescribir su infraestructura interna. Una vez que dos organismos respetan el mismo esquema, son interoperables por construcción. No hay que negociar ningún convenio adicional.

El cambio fundamental es pasar de intercambiar documentos a publicar hechos. Un organismo no emite un certificado en PDF — publica una afirmación verificable bajo su URL. Cualquier otro organismo que necesite ese dato no solicita un documento: consulta directamente la URL, verifica la firma, y obtiene un hecho procesable automáticamente. El permiso de lectura es implícito en la publicación. No hace falta una autorización específica para cada consulta.

Ejemplo. Un organismo necesita verificar que una persona tiene título universitario, máster y está colegiada. Hoy: tres consultas a tres sistemas distintos, tres formatos diferentes, tres autorizaciones, cruce manual de datos. Con Trusteando: tres URLs, mismo formato, misma verificación, resultado automáticamente coherente porque cada dato está firmado por quien tiene autoridad sobre él.

La capa de adaptadores (sección 11) permite que organismos con sistemas legados participen sin reescribir su infraestructura: el adaptador expone la misma interfaz Trusteando sobre cualquier base de datos interna. Para el verificador externo, consultar el registro civil de un ayuntamiento es idéntico a consultar el registro de titulaciones de una universidad. La complejidad interna queda encapsulada; la interfaz pública es uniforme.

Los organismos pequeños sin capacidad técnica propia pueden delegar la publicación en un nodo superior — una diputación provincial, un organismo agregador — que actúa como proveedor de infraestructura. Los datos se publican bajo la URL del organismo delegante, firmados de forma que se puede verificar su origen. La interoperabilidad no requiere que cada actor tenga infraestructura propia.

7.9 Acuerdos y contratos como nodos del grafo

Un acuerdo entre dos o más partes puede representarse como un nodo independiente en el grafo. Siguiendo la analogía OOP de la sección 2.11, el acuerdo es el objeto, los participantes son sus propiedades, y plan/execution son sus métodos. Cada parte publica el acuerdo en su propio espacio, y la concordancia entre ambas publicaciones constituye la prueba de consentimiento bilateral.

```
acuerdo-servicios-2026-xyz/
├── [state trusteando]/              ← object.state
├── participantes/                   ← object.collection
│   ├── [empresaA proveedor]/         ← object.property (inmutable)
│   └── [empresaB cliente]/           ← object.property (inmutable)
├── plan/                            ← object.method (intención)
│   ├── [objetivo "servicios consultoría 2026"]/
│   ├── [plazo "12 meses"]/
│   └── [state approved]/
├── execution/                       ← object.method (realidad
acumulada)
│   ├── firma/since/2026-03-18/
│   └── hito-1/[state completed]/since/2026-06-15/
└── private/             ← condiciones económicas, privadas
```

Cada parte publica una copia idéntica en su propio espacio:

```
empresaA.es/trusteando/acuerdos/acuerdo-servicios-2026-xyz/
empresaB.es/trusteando/acuerdos/acuerdo-servicios-2026-xyz/
```

Un verificador que consulte cualquiera de las dos URLs puede verificar la firma de esa parte, comprobar que la otra parte ha publicado la misma estructura, confirmar que las fechas de firma coinciden, y concluir que el acuerdo es bilateralmente válido. Si las estructuras difieren, hay disputa. Si una parte no ha publicado, el acuerdo no está completado. El protocolo no necesita ningún sistema externo para registrar contratos — la concordancia entre dos nodos es la prueba.

7.10 Bootstrapping de identidad mediante control de canal externo

El protocolo requiere habitualmente que un nodo padre otorgue una clave a un hijo mediante grant_key. Pero hay entidades que ya tienen presencia verificable en plataformas externas — Hacker News, GitHub, Twitter — sin tener un dominio propio. El mecanismo de bootstrapping mediante canal externo permite incorporar esas identidades al protocolo sin romper la jerarquía de confianza.

El flujo es el siguiente. El usuario solicita al nodo plataforma — por ejemplo trusteando/hn/ — unirse al sistema. El nodo genera un par de claves derivadas del identificador del usuario:

```
verification_key = node.grant_key(user_id + "verify")
private_key      = node.grant_key(user_id + "private")
```

El nodo envía verification_key al usuario. El usuario la publica en su perfil público de la plataforma — el campo about de HN, la bio de GitHub, la descripción de Twitter. El nodo consulta ese perfil, verifica que verification_key está ahí, y confirma la identidad. Solo quien controla el perfil puede publicar en él — esa es la prueba. A partir de ese momento el usuario puede responder retos con private_key mediante respond_to_challenge, y el nodo puede verificar con verify_child_authorship sin necesidad de almacenar nada — rederiva private_key en cada verificación a partir de user_id y su propio secreto.

La estructura del nodo plataforma en el grafo es la red paralela hecha visible: cada usuario verificado es un nodo hijo con su propia entrada, su fecha de incorporación, y su estado. El nodo no pertenece a Hacker News ni a ninguna plataforma — es un nodo independiente del protocolo que usa HN como mecanismo de verificación inicial.

```
trusteando/hn/
├── [source news.ycombinator.com]/
├── [quorum 1]/
└── registry/
    ├── [dang is-member]/
    │   ├── since/2026-03-19/
    │   └── [state trusteando]/
    ├── [pg is-member]/
    │   ├── since/2026-03-19/
    │   └── [state trusteando]/
    └── [confidencenode0 is-member]/
        ├── since/2026-03-19/
        └── [state trusteando]/
```

El mismo mecanismo es aplicable a cualquier plataforma con perfiles públicos y un campo editable: trusteando/github/, trusteando/twitter/, trusteando/linkedin/. Cada plataforma existente puede tener su nodo espejo verificable. Los usuarios que ya tienen presencia en esas plataformas tienen un camino de entrada al protocolo sin necesitar dominio propio. Y pueden vincular sus identidades de diferentes plataformas bajo una identidad autónoma única usando old_identities/. La red paralela no empieza desde cero — se superpone a las identidades que ya existen en la web.

7.10.1 Jerarquía natural expresada en código

La relación padre-hijo entre el nodo plataforma y sus usuarios es directamente expresable con la clase TrusteandoNode. Los sufijos :verify y :private garantizan que las dos claves derivadas son independientes — conocer una no revela nada sobre la otra:

```
hn_node = TrusteandoNode(key_hn)

# El nodo deriva el par para cada usuario
verification_key = hn_node.grant_key("dang:verify")   ← pública, se
publica en el perfil
private_key      = hn_node.grant_key("dang:private")  ← privada, la
usa el usuario

# El nodo verifica cualquier acción del usuario sin almacenar nada:
def verify_user_action(user_hn, context, proof):
    child_key = hn_node.grant_key(user_hn + ":private")
    return hn_node.reduce_hash(child_key, context) == proof
```

7.10.2 El camino desde el bootstrapping a la identidad autónoma

Un usuario que entra por una plataforma externa tiene inmediatamente una clave privada funcional. El recorrido natural es: usar la clave para responder retos y firmar contenido, acumular reputación bajo esa identidad, y cuando decida tener presencia propia migrar a un dominio con old_identities/ preservando el historial completo:

```
# Identidad inicial: derivada del nodo HN
old_identity = hn_node.grant_key("dang:private")

# Cuando obtiene dominio propio:
dang.com/trusteando/old_identities/
    └── [new_identity] → [old_hn_identity, ...]   ← historial
preservado
```

La reputación acumulada bajo la identidad HN no se pierde — se transfiere. El protocolo no obliga a empezar de cero cuando se da el salto a la identidad autónoma.

7.10.3 Propiedades del mecanismo

Sin registro nuevo: el usuario usa su cuenta existente en la plataforma. Prueba de control: publicar verification_key demuestra control del perfil — solo quien lo controla puede publicar ahí. Sin almacenamiento: el nodo puede rederivir claves cuando las necesita, no guarda estado. Jerarquía natural: el nodo de la plataforma es padre de los usuarios, exactamente el mismo modelo que el resto del protocolo. Privacidad: private_key nunca se revela, ni siquiera al nodo — el usuario responde retos sin exponerla. Uniformidad: no hay caso especial para bootstrapping, es la misma función grant_key con distintos sufijos. Portabilidad futura: el usuario puede migrar a un dominio propio usando old_identities/, preservando el historial completo y la reputación acumulada.

8. Modelo de sostenibilidad

El protocolo es abierto. La implementación de referencia es software libre. Lo que puede monetizarse es la capa de servicio sobre el protocolo — no el protocolo mismo.

8.1 Reconocimiento formal como agente

El registro en el root de Trusteando tiene valor de señal, especialmente en las fases iniciales cuando el root es la principal raíz de confianza establecida. El reconocimiento formal tiene un coste razonable y criterio público y transparente.

8.2 Infraestructura gestionada

Implementar un nodo Trusteando completo requiere conocimientos técnicos que no todas las entidades tienen. Trusteando ofrece infraestructura gestionada — el nodo como servicio — para entidades que quieren participar sin gestionar la infraestructura. Es el modelo de WordPress: el protocolo es libre, el hosting gestionado no tiene por qué serlo.

8.3 API de verificación

Aplicaciones de terceros que quieran integrar verificación Trusteando sin implementar el protocolo completo pueden usar una API de verificación. El precio es por volumen de verificaciones. El protocolo permanece libre — la API es una conveniencia, no un requisito.

8.4 Lo que permanece libre

- El formato completo del protocolo
- La capacidad de montar nodos propios
- La verificación manual consultando URLs directamente
- La publicación de credenciales sin pasar por el root
- La implementación de verificadores propios

9. El problema del arranque

El valor del root depende de la reputación de quien lo opera. Los sistemas de confianza no nacen de la nada — necesitan un punto de partida con credibilidad suficiente para que las primeras entidades quieran registrarse.

Este es el mismo problema que resolvieron DNS, Let's Encrypt, y cualquier PKI: al principio la autoridad no viene del sistema — viene de las personas y organizaciones detrás de él. La solución no es técnica sino social: las primeras entidades reconocidas se consiguen con relaciones personales, casos de uso concretos que resuelven problemas reales, y credibilidad del equipo fundador en ese contexto.

La estrategia de arranque correcta es publicar el protocolo abiertamente y con prioridad temporal establecida, antes de que ningún competidor pueda reclamar la misma idea. La apertura no debilita la posición del root — la refuerza. Un root que opera un protocolo cerrado puede ser reemplazado por cualquiera que publique un protocolo mejor. Un root que opera el protocolo estándar de facto acumula legitimidad que no se puede copiar.

Sin embargo, una sola entidad operando el root inicial introduce un riesgo que el diseño técnico por sí solo no puede eliminar: la coacción. Aunque el root no tiene discrecionalidad — solo ejecuta un algoritmo público — la entidad que opera el servidor sí puede ser objeto de presión legal, política o regulatoria. Un gobierno puede ordenar a una fundación que excluya ciertos nodos. Una empresa puede ser adquirida. Un individuo puede ser intimidado. La solución técnica a este riesgo es aplicar desde el día 1 el mismo mecanismo de quórum definido en la sección 4.10.

El sistema no arranca con un root único, sino con un conjunto fundador: N entidades independientes — universidades, ONGs, organismos técnicos, administraciones públicas — que firman un acuerdo de génesis y operan cada una su propio nodo root desde el primer día. Cualquier operación sensible requiere el quórum de K de esos N nodos fundadores. Ningún nodo fundador tiene autoridad unilateral. El sistema nace ya siendo policentríco, no lo alcanza después de un período de maduración bajo una autoridad central transitoria.

La ventaja adicional es que los nodos fundadores aportan reputación institucional diversa desde el arranque. Un verificador que confíe en cualquiera de ellos tiene acceso inmediato al sistema completo. La resistencia a la coacción es proporcional a la diversidad jurídica y geográfica del conjunto fundador: para silenciar el sistema un atacante necesita comprometer simultáneamente el quórum completo de entidades en múltiples jurisdicciones.

9.1 Resistencia a Embrace, Extend, Extinguish

La estrategia conocida como Embrace, Extend, Extinguish consiste en adoptar un estándar abierto, añadir extensiones propietarias que crean dependencia, y finalmente hacer irrelevante el estándar original. Es el patrón que ha eliminado o debilitado múltiples protocolos abiertos a lo largo de la historia de internet.

Trusteando incorpora resistencia a este patrón por diseño:

- Embrace — cualquiera puede adoptar el protocolo. Inevitable y deseable.
- Extend — cualquier extensión propietaria debe declararse bajo trusteando/extensions/ con justificación. Las extensiones no estándar no son interpretadas por otros parsers. La fragmentación es visible y auditable.
- Extinguish — los datos viven en la URL de cada entidad, no en ningún servicio central. No hay nada que apagar. GPL v3 impide distribuir versiones cerradas. El root no guarda estado — no se puede monopolizar.

La defensa más efectiva contra este patrón es cubrir el espacio funcional tan completamente que cualquier extensión propietaria sea marginal o redundante. El vocabulario extenso de carpetas estándar — features/, payments/, messaging/, hooks/, media/ — no es complejidad innecesaria: es territorio que no se puede usar como cuña.

9.2 Impacto en servicios intermediarios

Un efecto secundario del protocolo es que la información estructurada de cualquier negocio — horarios, ubicación, características, reputación verificada — pasa a ser pública y accesible directamente desde su URL. Esto permite construir servicios de búsqueda, comparación, y agregación sobre datos abiertos sin depender de intermediarios que actualmente centralizan y monetizan esa información.

El protocolo no elimina esos servicios — simplemente hace que su valor resida en la capa de presentación y experiencia de usuario, no en el acceso exclusivo a los datos. Los hooks y proxies del protocolo garantizan además que el negocio nunca esté atado a ningún proveedor de servicios por razones técnicas — cambiar de proveedor es siempre una decisión de negocio, nunca una migración técnica.

10. Dos capas — registro y trámite

El protocolo distingue explícitamente dos capas que en los sistemas actuales se confunden:

10.1 La capa de registro — el Knowledge Graph

La capa de registro es el grafo permanente de hechos verificados. Lo que es cada nodo y qué relaciones tiene. Cada entrada es firmada por la entidad que la publica, inmutable una vez establecida, y auditable por cualquier tercero sin cooperación de nadie.

```
universidad.es/profesores/juan-ruiz          ← hecho verificado,
permanente
estados/españa/ciudadanos/juan-ruiz          ← hecho verificado,
permanente
colegios/medicos/colegiados/juan-ruiz        ← hecho verificado,
permanente
```

La redundancia en esta capa es una propiedad positiva — que varios nodos independientes certifiquen lo mismo sobre Juan añade resiliencia. Si dos nodos de alto grado dicen cosas contradictorias, esa inconsistencia es una señal relevante por sí misma.

10.2 La capa de trámite — los flujos entre nodos

La capa de trámite son los intercambios de información entre nodos para llegar a establecer hechos en la capa de registro. Son efímeros, operacionales, y no forman parte del grafo permanente. Representan el proceso, no el resultado.

```
Universidad pregunta a estado: ¿juan-ruiz tiene DNI válido?   ← flujo,
efímero
Estado responde: sí, expedido 2015                            ← flujo,
efímero
Universidad añade juan-ruiz a /profesores/since/2026-03-17/   ← entra
al grafo
```

El protocolo puede definir el formato estándar de estos flujos para facilitar la interoperabilidad entre implementaciones. Pero los flujos no son el grafo — son las aristas operacionales que permiten construirlo. Un verificador externo nunca necesita ver los flujos, solo el resultado en el grafo.

10.3 Identidad civil como condición mínima

Un nodo puede existir técnicamente sin identidad civil verificable. Pero para que un nodo represente a una persona física real, el protocolo requiere al menos una credencial emitida por una entidad con autoridad civil reconocida — un estado, un registro oficial — que vincule ese nodo con una identidad legal.

La estructura natural para personas físicas es:

```
estados/españa/ciudadanos/juan-ruiz     ← ancla de identidad civil
  → universidad certifica: juan-ruiz es Profesor
  → colegio certifica: juan-ruiz está colegiado
  → empresa certifica: juan-ruiz es empleado
```

El nodo de identidad civil es el ancla. Las credenciales son aristas del grafo que apuntan a ese nodo desde distintas entidades. Una persona puede tener múltiples credenciales de múltiples entidades — todas apuntan al mismo nodo de identidad civil.

11. Especificación declarativa e implementación funcional

El protocolo distingue dos niveles que son independientes y complementarios:

11.1 La especificación declarativa — las carpetas

Las carpetas definen qué existe y qué significa. Es el contrato público del protocolo — el QUÉ. Cualquiera puede leerlo, verificarlo, y auditarlo sin herramientas especiales ni conocimiento de la implementación interna. La estructura de carpetas es permanente, firmada, y auditable.

11.2 La implementación funcional — la base de datos

La base de datos, la API, el adaptador definen cómo se almacena y recupera internamente — el CÓMO. Es un detalle de implementación invisible para el verificador externo. Cualquier implementación funcional es válida si produce resultados indistinguibles de leer las carpetas directamente.

```
# Acceso declarativo — web estática
GET turismo.gob.es/establecimientos/casapepe.es/classification/stars
→ 3

# Acceso funcional — base de datos
query({ path: 'establecimientos/casapepe.es/classification/stars' })
→ 3

# El resultado es idéntico — el parser no sabe ni le importa cómo se
almacena
```

Es el mismo principio que separa una interfaz de su implementación en programación orientada a objetos. La interfaz es declarativa — define qué existe y qué devuelve. La implementación es funcional — define cómo lo hace internamente.

11.3 El sistema de adaptadores

Un organismo como un ministerio tiene sus datos en una base de datos existente. No necesita reescribirla para participar en el protocolo — solo necesita publicar una función adaptadora que responde a rutas del protocolo con los mismos valores que tendría una carpeta.

El organismo publica en su nodo:

```
turismo.gob.es/trusteando/adapters/clasificacion_hotelera/
  name           ← 'clasificacion_hotelera'
  description    ← 'Verifica clasificación oficial de
establecimientos'
  input_schema   ← qué campos del protocolo necesita como entrada
  output_schema  ← qué campos devuelve al protocolo
  version
  cache/ttl      ← con qué frecuencia actualizar
```

La función adaptadora tiene una firma estándar:

```
async function query(path) {
  // path: ruta del protocolo
```

```
  // 'establecimientos/casapepe.es/classification/stars'

  // internamente consulta su base de datos
  // SELECT stars FROM establecimientos WHERE url = 'casapepe.es'

  // devuelve como si fuera una carpeta
  return { value: 3, timestamp: '2024-01-15' }
}
```

11.4 El repositorio central de adaptadores

Si el organismo no publica su adaptador, el repositorio central de ConfidenceNode mantiene adaptadores de comunidad para fuentes públicas conocidas. Cuando el organismo publique el suyo, tiene precedencia automáticamente — el parser siempre prefiere el adaptador del propio nodo.

```
confidencenode.org/trusteando/adapters/
  turismo.gob.es/clasificacion_hotelera/   ← mantenido por la
comunidad
  guia.michelin.com/estrellas/              ← mantenido por la
comunidad
```

11.5 La migración gradual

Un organismo puede moverse a su ritmo sin interrumpir el servicio:

```
Fase 1 — sin adaptador
  Parser lee carpetas estáticas si existen, o usa adaptador de
comunidad

Fase 2 — adaptador sobre base de datos existente
  Organismo publica query() sobre su BD actual — sin cambiar nada más

Fase 3 — web nativa del protocolo
  Organismo publica carpetas reales — el adaptador ya no es necesario
  La ruta funciona igual para todos los parsers existentes
```

La especificación declarativa es el destino. La implementación funcional es el camino. El protocolo funciona en los tres estados sin cambiar nada en los parsers que ya lo usan.

12. Cuestiones abiertas

Las siguientes cuestiones están identificadas pero deliberadamente fuera del alcance de esta versión del protocolo. Se documentan aquí para establecer que son conocidas y para orientar el trabajo futuro.

12.1 Fallo de una entidad ancla

Si la URL de una entidad desaparece de forma abrupta — quiebra repentina, pérdida del dominio — las credenciales que emitió quedan sin ancla. Para desapariciones ordenadas el protocolo facilita la transferencia de registros a un sucesor. Para desapariciones abruptas la solución es externa al protocolo — institucional o legal. El protocolo hereda ese límite honestamente.

12.2 Ámbito de las credenciales emitidas

El protocolo no impone restricciones sobre qué puede certificar un agente reconocido. Los mecanismos para declarar y verificar el ámbito de competencia de un agente se diseñarán en v0.3.

12.3 Privacidad avanzada mediante pruebas de conocimiento cero

Una implementación basada en zero-knowledge proofs permitiría demostrar la validez de una credencial sin revelar ningún elemento de la cadena. Esta es una dirección de investigación para versiones futuras.

12.4 Formato estándar de la capa de trámite

El protocolo define la capa de registro pero no el formato de los flujos entre nodos en la capa de trámite. Un estándar de interoperabilidad para estos flujos se abordará en v0.2.

12.5 Sistema de certificación de compatibilidad

Una herramienta de testing que evalúa si un nodo usa campos estándar para funciones estándar, o ha inventado equivalentes sin justificación. Los nodos pueden declarar extensiones justificadas bajo trusteando/extensions/ con una rationale y un organismo certificador. Un sistema de organismos de valoración y resolución de disputas sobre extensiones está previsto para v0.4.

12.6 Sandboxing de adaptadores

Los adaptadores ejecutan código de terceros. El protocolo debe especificar el entorno de ejecución restringido — sin acceso a red salvo a la URL del organismo, sin efectos secundarios, con timeout máximo. Web Workers y Content Security Policy son la base técnica. La especificación formal se abordará en v0.2.

12.7 Privacidad del grafo de relaciones

La divulgación selectiva definida en la sección 6 protege el contenido de las credenciales, pero la existencia de la arista en el grafo es pública por defecto: que universidad.es/profesores/juan-ruiz sea una URL pública revela que Juan tiene una relación con esa universidad. Este problema está resuelto mediante la carpeta private/ definida en la sección 2.11. Publicar una entrada bajo universidad.es/personal/profesores/private/ oculta la identidad del nodo — un verificador sabe que algo existe ahí sin saber qué. Para acceder debe elevar una consulta al agente que controla la carpeta. La existencia de la arista puede ocultarse, no solo su contenido. Las pruebas de conocimiento cero (sección 12.3) complementarán este mecanismo en versiones futuras para casos donde ni siquiera la existencia de private/ deba ser visible.

12.8 Marco legal de resolución de disputas

El Apéndice B define un mecanismo de disputas con evidencia criptográfica y costes crecientes, pero no especifica cómo se vinculan las resoluciones del root o de los árbitros con el sistema judicial real. El hash_disputa es una prueba de identidad ante el árbitro, pero el árbitro necesita un marco procesal para tomar decisiones vinculantes sobre el significado de una credencial o la legitimidad de una relación. Cómo se integra Trusteando con marcos jurísdicos concretos — más allá del uso de eIDAS para la firma — es una cuestión abierta que requerirá colaboración con expertos legales en cada jurisdicción.

12.9 Fragmentación semántica del vocabulario

La gramática de rutas define la sintaxis pero no el vocabulario. La apertura del vocabulario de operadores y propiedades — cualquier término en lenguaje natural es válido — es una fortaleza expresiva pero puede generar fragmentación semántica: diferentes comunidades o dominios podrían usar términos distintos para el mismo concepto, reduciendo la interoperabilidad que el protocolo promete. Un nodo que use [en] y otro que use [located-in] para la misma relación geográfica no son automáticamente interoperables. La solución a largo plazo pasa por ontologías de dominio o registros de términos consensuados por comunidad, algo que el protocolo facilita pero no especifica. El vocabulario reservado del Apéndice A es el núcleo común; su extensión coordinada es una cuestión de gobernanza más que de especificación técnica. Una dirección concreta es publicar ontologías por dominio en repositorios distribuidos alojados en múltiples roots, donde comunidades de práctica consensuan los términos estándar para su sector. Las herramientas de análisis pueden advertir al operador de un nodo cuando usa un término que diverge del vocabulario mayoritario en su dominio, fomentando la convergencia sin imposición. La especificación formal de este mecanismo se abordará en versiones futuras.

12.10 Descubrimiento y registro de nombres

El protocolo identifica entidades por hash de URL, no por nombre. La asociación entre un nombre legible — "Universidad de Málaga" — y su URL uma.es es un paso externo al protocolo que en la versión actual queda sin especificar. Un nodo puede publicar su nombre oficial como propiedad: [nombre-oficial "Universidad de Málaga"]/[indicates]/[source uma.es]. Roots y nodos de alta reputación pueden atestiguar esa asociación publicando su propia versión del hecho, creando un registro de nombres descentralizado donde la confianza en la asociación es una cuestión de grado resoluble mediante el mecanismo de quórum del protocolo. La especificación de este mecanismo se abordará en versiones futuras.

12.11 Autenticación activa con rotación de clave

El protocolo define la clave de emergencia para migraciones, pero no especifica un mecanismo de autenticación activa interactiva. Una extensión natural es la rotación atómica: el wallet genera una nueva clave y la publica en la web antes de revelar la antigua al verificador. El flujo es: (1) wallet genera clave_2 y publica hash_publico_2, (2) wallet espera confirmación de que hash_publico_2 está activo, (3) wallet revela clave_1 al verificador, (4) verificador comprueba hash(entity_id, clave_1) == hash_publico_1. Cada autenticación consume la clave revelada, convirtiendo el sistema en un mecanismo de contraseñas de un solo uso criptográficamente anclado en la identidad del nodo. La especificación formal de este mecanismo se abordará en versiones futuras.

12.12 Fuga de metadatos en la estructura de rutas

La carpeta private/ oculta el contenido de una relación pero no necesariamente su existencia ni el tipo de relación. Una ruta como empresa.com/trusteando/fusiones-y-adquisiciones/private/ revela información sensible aunque el contenido esté protegido — el nombre de la carpeta es ya una filtración. La solución es usar nombres opacos para carpetas sensibles: una carpeta cuyo nombre es un identificador no descriptivo no revela nada sobre su contenido. El protocolo permite esto sin ninguna extensión — el propietario elige el nombre de sus carpetas. La convención 1234/[folder-has-private-name]/ (pendiente de especificación formal) declara explícitamente esta intención para que las herramientas la traten correctamente.

12.13 Latencia en revocación de seguridad crítica

El modelo de réplicas observadoras con umbral de histéresis (sección 4.3) introduce una ventana de inconsistencia entre el momento en que una credencial es revocada y el momento en que todas las réplicas reflejan esa revocación. Para casos de uso de baja criticidad esta latencia es aceptable. Para control de acceso en tiempo real — acceso físico a instalaciones, autorizaciones de sistemas críticos — puede ser peligrosa. Una dirección de solución es un canal de revocación de alta prioridad: las réplicas sincronizan las revocaciones publicadas en registry/compromised/ con prioridad máxima sobre cualquier otra actualización, reduciendo la ventana de inconsistencia de minutos a segundos. La especificación formal de este mecanismo se abordará en versiones futuras.

12.14 Recuperación social de identidad

El mecanismo de custodia de secretos mediante wallets colaboradores (sección F.1) aborda la pérdida de claves para el autor del protocolo, pero no especifica un mecanismo genérico para cualquier entidad. Sin recuperación social, la pérdida de la clave de emergencia de un nodo es irreversible: su historial queda huérfano permanentemente. La dirección natural es el Secreto Compartido de Shamir: fragmentar la clave entre N guardianes de confianza de forma que cualquier subconjunto de M de ellos pueda reconstruirla. Los guardianes son entidades del propio grafo — universidad, empleador, registro oficial — nombradas por el nodo usando la convención [agent role]. La especificación formal de este mecanismo se abordará en versiones futuras.

12.15 Carga de verificación en cadenas profundas

Verificar un hecho al final de una cadena larga — root, universidad, facultad, departamento, profesor — requiere múltiples peticiones HTTP y el procesamiento de varios HMACs y firmas ECDSA en secuencia. Esto penaliza a dispositivos con recursos limitados o conexiones lentas. La dirección de solución es la agregación de pruebas mediante zero-knowledge proofs (sección 12.3): una única prueba ZK que demuestra la validez de toda la cadena sin revelar los pasos intermedios reduce la carga de verificación a una única operación, independientemente de la profundidad de la cadena. Una solución más inmediata y sin ZK es que cada nodo publique una prueba de Merkle precomputada de su posición en la cadena, permitiendo verificación local sin consultas adicionales.

---

13. Hoja de ruta

Las versiones futuras del protocolo se guían por los problemas que la práctica de implementación revele como más urgentes. La hoja de ruta siguiente refleja el estado actual de las prioridades conocidas — no es un compromiso de fechas sino una declaración de intención.

v0.1 — actual

Fundamentos del protocolo: URL como ancla de identidad, separación autenticidad/autoría, clave de emergencia y portabilidad, vocabulario reservado, capa de negocio, dos niveles de existencia, resistencia a Embrace–Extend–Extinguish.

v0.2

Extensiones semánticas: gramática de rutas con operadores relacionales entre corchetes, convención plan/execution para procesos activos. Casos de uso ampliados: coordinación de emergencias multi-agencia, interoperabilidad administrativa. Escalabilidad HMAC:

verificación por lotes mediante árboles de Merkle. Especificación formal de sandboxing de adaptadores y estándar de interoperabilidad para la capa de trámite. Wallet de referencia: agente de usuario básico que gestiona claves de forma segura, presenta credenciales gráficamente y permite compartir y verificar credenciales sin conocimientos técnicos. El wallet no es parte del protocolo — es la capa de aplicación que lo hace accesible al usuario final, análoga a un navegador web respecto a HTTP.

v0.3

Mecanismos de ámbito de credenciales: declaración y verificación del ámbito de competencia de un agente. Privacidad avanzada mediante pruebas de conocimiento cero para verificación sin interacción en línea. Implementaciones de referencia en al menos dos lenguajes.

v0.4

Sistema de organismos de valoración y resolución de disputas sobre extensiones. Mecanismo formal de rotación o sustitución del root por consenso de nodos de alto grado. Hoja de ruta hacia criptografía resistente a computación cuántica.

v1.0

Protocolo estable. Ecosistema de herramientas: validador público, generador asistido, tutoriales interactivos. Red con suficientes nodos de alto grado para operar de forma policéntrica sin dependencia del root en el uso cotidiano.

FAQ

¿Por qué URLs y no DIDs o W3C Verifiable Credentials?

Los DIDs (Decentralised Identifiers) del W3C requieren infraestructura adicional — registros blockchain, métodos DID específicos — y no están anclados en ninguna presencia real preexistente. Las W3C Verifiable Credentials definen el formato del documento pero no el sistema de confianza ni cómo se establece la autoridad del emisor. Trusteando usa URLs porque las entidades ya existen con URLs propias — no hace falta crear ninguna infraestructura nueva para identificarlas. La URL es la identidad, no un puntero a ella.

¿Qué pasa si una entidad pierde su dominio?

El protocolo tiene dos niveles de anclaje. En el nivel básico, la identidad está anclada en la URL y su pérdida abrupta es un riesgo real, mitigable con mirrors y dominios alternativos. En el nivel avanzado (sección 4.10), la entidad puede desacoplar completamente su identidad de la URL mediante secretos autónomos generados localmente. En ese caso la URL es solo el lugar de publicación: la identidad sobrevive a cambios de dominio, caídas de servidor y bloqueo DNS. Las dos modalidades son compatibles y vinculables mediante old_identities/.

¿Puede cualquiera crear un nodo?

Sí. Cualquier entidad con una URL puede publicar su espacio de identidad siguiendo el protocolo sin permiso de nadie. Esto crea un nodo en nivel 1 — identidad declarada, verificable por contexto. Para alcanzar nivel 2 — identidad reconocida con verificabilidad criptográfica por terceros desconocidos — hace falta el reconocimiento del root o de otro nodo de confianza suficiente.

¿Qué diferencia hay entre esto y PGP Web of Trust?

PGP Web of Trust es una red de firmas entre personas donde la confianza se propaga por conocidos. No tiene estructura jerárquica, no está anclado en URLs públicas, no tiene dimensión temporal verificable, y no distingue entre tipos de relación. Trusteando es un Knowledge Graph estructurado donde las entidades son nodos con URLs propias, las relaciones tienen semántica explícita expresada en la estructura de carpetas, y el historial completo es auditable. PGP prueba que alguien conoce a alguien. Trusteando prueba qué es cada entidad y desde cuándo.

¿Es el root de Trusteando un punto de control centralizado?

El root es el nodo de arranque con mayor reputación inicial. No guarda estado — solo ejecuta verificaciones algorítmicas sobre información que las propias entidades publican. Si el root cae, cualquier nodo de alto grado puede realizar las mismas verificaciones porque el algoritmo es público. Con el tiempo las entidades con reputación suficiente se convierten en raíces de confianza independientes. El root es necesario para arrancar el sistema — no para que funcione una vez establecido.

¿Cómo se relaciona Trusteando con eIDAS?

eIDAS es el reglamento europeo de identidad digital — define los requisitos legales para que una firma electrónica tenga validez jurídica. Trusteando establece la evidencia criptográfica; eIDAS le da fuerza legal en jurisdicciones europeas. Son complementarios. Un nodo Trusteando respaldado por una entidad reconocida bajo eIDAS hereda su validez legal automáticamente.

¿La verificación requiere consultar al superior cada vez? ¿Escala bien?

El diseño base requiere consultar al superior para verificar el HMAC, pero hay tres mecanismos de escalabilidad. Caché firmada: el superior publica listados firmados de sus miembros con TTL, permitiendo verificaciones offline durante ese período. Árboles de Merkle: el superior publica una raíz de Merkle de todos los HMACs válidos; el miembro presenta una prueba junto con su credencial, permitiendo verificación sin consulta online. Múltiples roots y quórum: para credenciales de alto valor se puede requerir validación por un quórum de roots independientes (sección 4.10.4), distribuyendo la carga. Los tres mecanismos están descritos en la sección 4.3. La operación de verificación subyacente en todos ellos es la función verify_child_authorship de la clase TrusteandoNode (sección 4.11).

Un pequeño negocio no puede gestionar claves y carpetas. ¿Cómo se adopta?

Hay tres caminos de adopción. Hosting gestionado: igual que WordPress, el protocolo es libre pero cualquiera puede contratar a alguien para que gestione su nodo (sección 8.2). Migración asistida por LLM: el usuario describe dónde tiene sus datos y un modelo de lenguaje genera automáticamente la estructura completa; el humano solo revisa y valida. Adaptadores: organismos con bases de datos existentes pueden publicar un adaptador que traduzca consultas del protocolo a su sistema interno sin migrar datos (sección 11). En los tres casos la complejidad técnica queda fuera del alcance del usuario final.

¿No es todo público? ¿Qué pasa con la privacidad?

El protocolo ofrece tres niveles de privacidad selectiva (sección 6). Hash del mensaje: solo se publica el hash, no el contenido; se puede verificar integridad sin revelar el mensaje. Divulgación selectiva: el titular puede elegir revelar solo el resultado, solo el emisor, o la

cadena completa según el contexto. Carpetas private/: información que existe pero no se expone por defecto, accesible solo mediante grantReveal(). La privacidad no es una capa añadida — está diseñada dentro de la estructura desde el principio.

¿Y si un root es comprometido? ¿Puede falsear identidades?

Un root comprometido puede falsear las claves que genera para nuevas entidades, pero no puede falsear el pasado: las credenciales ya emitidas están firmadas por las entidades, no por el root; las relaciones ya establecidas están publicadas en los nodos, no en el root; y el quórum de múltiples roots hace que comprometer uno solo no sea suficiente para operaciones que requieren consenso. Además, un root comprometido pasa a state brokenado — término del vocabulario reservado del protocolo — visible para todos los participantes, y sus firmas dejan de contar para el quórum.

El protocolo parece muy ambicioso. ¿Puede funcionar en la práctica?

Trusteando no intenta reemplazar todos los sistemas de identidad existentes de golpe. Su estrategia es orgánica y por capas. Nivel 1 declarado: cualquier entidad puede publicar su espacio sin permiso, útil como prueba de concepto o para comunidades cerradas. Nivel 2 reconocido: entidades que necesitan verificabilidad por terceros desconocidos solicitan reconocimiento a roots. Adopción sectorial: el protocolo puede empezar en nichos concretos — restauración, emergencias, universidades — y crecer desde ahí. El protocolo no necesita conquistar el mundo para ser útil. Basta con que resuelva problemas reales en dominios concretos, y eso ya es posible hoy con la especificación actual.

Apéndice A — Vocabulario reservado del protocolo

Las siguientes carpetas tienen semántica específica en el protocolo. Cualquier implementación debe respetar estos nombres y su significado. La lista es extensible en versiones futuras — ningún nombre reservado puede ser redefinido por una implementación particular.

A.0 La carpeta raíz — trusteando/

Todo lo que el protocolo necesita vive bajo una única carpeta raíz: trusteando/. Es la única carpeta que una entidad necesita crear, mover, o subir para participar en el protocolo. Si la entidad cambia de hosting manteniendo el mismo dominio, mueve la carpeta trusteando/ al nuevo servidor y todo sigue funcionando — el entity_id no cambia porque se calcula sobre el dominio normalizado, no sobre la infraestructura.

```
casapepe.es/trusteando/     ← todo el protocolo vive aquí
  identity/
  registry/
  transactions/
```

A.1 Normalización de URL para el cálculo de entity_id

El entity_id se calcula externamente por cualquier verificador — la entidad no puede declararlo ni influir en él. Esto es una consecuencia directa de la regla de oro: la estructura declara información pero no puede imponer campos del protocolo.

La normalización es mínima y solo elimina campos técnicos del esquema URL — nunca paths, nunca componentes semánticos. Esto es crítico para evitar colisiones: si se eliminaran paths, dos entidades distintas podrían obtener el mismo entity_id.

```
normalizar(url):
  1. minúsculas
  2. eliminar prefijo www
  3. eliminar puerto estándar (443 para https, 80 para http)
  4. eliminar trailing slash
  5. eliminar query params (?...)
  6. eliminar fragmentos (#...)
  7. conservar el path completo  ← nunca eliminar paths

entity_id = SHA-256(normalizar(url))

Ejemplos:
  https://WWW.CasaPepe.ES:443/  →  https://casapepe.es
  https://casapepe.es/?ref=google  →  https://casapepe.es
  https://casapepe.es/#inicio  →  https://casapepe.es
  https://banco.es/santander/  →  https://banco.es/santander
  https://banco.es/mio/  →  https://banco.es/mio
```

Edge case documentado: si la normalización eliminara paths, 'banco.es/santander/' y 'banco.es/mio/santander/' podrían colapsar al mismo entity_id. La regla de conservar paths completos elimina esta posibilidad. Dos entidades con distinto path son siempre entidades distintas.

A.2 trusteando/identity/

Contiene todo lo que el nodo declara sobre sí mismo:

```
trusteando/identity/
  public_key          ← clave pública del nodo (base64, curva P-256)
  hash_migracion      ← hash(entity_id, clave_emergencia_1)
                        usado para migración voluntaria a nuevo
superior
                        se renueva tras cada migración completada
  hash_disputa        ← hash(entity_id, clave_emergencia_2)
                        usado para resolución de disputas
                        nunca se revela en migraciones normales
  entity_type         ← universidad | empresa | persona | servidor
| ...
  scope               ← ámbito declarado de las credenciales que emite
  root_certificate    ← certificado firmado por el root (si existe)
  dispute_level       ← nivel máximo al que este nodo puede elevar
disputas
```

A.3 trusteando/registry/

Contiene todo lo que el nodo certifica sobre otros:

```
trusteando/registry/
  old_identities/     ← cadena histórica de identidades anteriores
                        mantenida por el superior receptor, no por el
nodo
                        contiene la cadena completa, no solo el
vínculo inmediato
```

```
  since/              ← fechas de incorporación de nodos reconocidos
  until/              ← fechas de baja de nodos reconocidos
  externals/          ← vínculos a sistemas externos no controlados
```

A.4 La carpeta externals/

Cualquier carpeta bajo externals/ declara que el nodo vincula algo que no controla. La semántica es opuesta al resto de carpetas — fuera de externals/, controlar la URL implica controlar el contenido. Dentro de externals/, el nodo solo garantiza la autenticidad del vínculo, no el contenido del destino.

A.5 Convención since/ y until/

Las fechas de vigencia de una credencial se expresan como subcarpetas con formato ISO 8601 (YYYY-MM-DD). La presencia en since/ sin entrada correspondiente en until/ indica que la credencial sigue vigente. Ambas carpetas están bajo control del superior, nunca del nodo certificado. La convención se extiende con dos variantes adicionales. from/ indica una condición de activación futura cuyo momento exacto no es una fecha conocida sino un evento: from/[time secure-wallet-is-ready]/ activa la carpeta cuando el wallet esté listo, no en una fecha fija. calculate/maximum/ y calculate/minimum/ son condiciones compuestas: maximum requiere que todas las subcarpetas estén presentes (AND lógico), minimum requiere que al menos una lo esté (OR lógico). Estas convenciones son especialmente útiles para expresar condiciones de lanzamiento y activación condicional de roles.

A.6 trusteando/identity/contact/ — opcional, recomendado para negocios

Campos de contacto público. Si existe la carpeta, los campos siguientes son los estándar reconocidos — usar campos no estándar para la misma función reduce la interoperabilidad:

```
trusteando/identity/contact/
  email                   ← RECOMENDADO
  phone                   ← RECOMENDADO
  web                     ← URL de contacto o formulario
  address/                ← dirección postal estructurada
    street
    number
    city
    region
    country               ← ISO 3166-1 alpha-2 (ES, FR, DE...)
    postal_code
  social/                 ← presencias sociales
    twitter
    linkedin
    mastodon
  languages               ← idiomas en que atienden
```

A.7 trusteando/identity/location/ — opcional, recomendado para negocios físicos

```
trusteando/identity/location/
  lat                     ← OBLIGATORIO si existe location/
  lon                     ← OBLIGATORIO si existe location/
  place_id/
```

```
    externals/          ← IDs en sistemas externos — no controlados
      google_maps
      osm               ← OpenStreetMap
      foursquare
  area/
    radius_km           ← área de servicio para entregas etc.
```

A.8 trusteando/identity/classification/ — opcional

Clasificación oficial del negocio. Si certified_by apunta a un nodo Trusteando, la verificación es criptográfica. Si apunta a una URL externa, se usa verifyExternal(). Si va bajo externals/, solo es un enlace sin verificación automática.

```
trusteando/identity/classification/
  [organismo]/          ← una entrada por organismo certificador
    stars               ← entero 1-5
    category            ← esquema alfanumérico alternativo
    certified_by        ← URL del nodo o página que certifica
    since               ← fecha de la clasificación
    until               ← fecha de expiración (si aplica)
    calculated/
      verified          ← bool — verificado contra la fuente
      last_checked
      cache/ttl
```

A.9 trusteando/status/ — opcional, recomendado para negocios con horario

Estado en tiempo real. Los campos de status/ tienen TTL corto por defecto. open es siempre calculated/ — se deriva de open_at_today y close_at_today.

```
trusteando/status/
  open_at_today         ← hora apertura hoy (HH:MM) — para clientes de
hoy
  close_at_today        ← hora cierre hoy (HH:MM) — para clientes de
hoy
  schedule/             ← horario habitual — para reservas futuras
    monday/ ... sunday/
      open_at
      close_at
    exceptions/         ← festivos, vacaciones, eventos
      [YYYY-MM-DD]/
        open_at
        close_at
        closed          ← bool explícito
        note            ← 'Semana Santa', 'Vacaciones agosto'
  occupancy/
    current             ← ocupación actual (0-100%)
    capacity            ← aforo máximo
    cache/ttl           ← corto — dato en tiempo real
  queue/
    current_number      ← número que se atiende ahora
    last_issued         ← último número emitido
    wait_minutes        ← espera estimada
    cache/ttl
  special/              ← ofertas o eventos activos
    [offer_id]          ← referencia a transactions/trusteando/offers/
  calculated/
```

```
    open                  ← bool: now >= open_at_today && now <
close_at_today
    open/cache/
      ttl                 ← segundos hasta próximo cambio de estado
```

A.10 trusteando/notifications/ — opcional

Canal de notificaciones para parsers y agregadores. Permite mantenerse sincronizado sin recorrer el árbol completo.

```
trusteando/notifications/
  last_updated            ← timestamp del último cambio
  changelog/
    [timestamp]/
      path                ← qué carpeta cambió
      action              ← created | updated | deleted | calculated
      summary
  cache/ttl               ← corto — los parsers comprueban
frecuentemente
```

A.11 Convención cache/ — aplicable a cualquier carpeta

Cualquier carpeta puede declarar su comportamiento de caché con una subcarpeta cache/. Si no existe, se aplica el TTL por defecto del protocolo según el tipo de contenido.

```
property/cache/
  ttl                     ← segundos hasta expiración (0 = no cachear)
  strategy                ← no_cache | short | normal | long

TTL por defecto según contenido:
  status/calculated/      → no_cache (0s)
  status/occupancy/       → corto (60s)
  transactions/offers/    → corto (300s)
  identity/               → largo (86400s)
  registry/               → normal (3600s)
  notifications/          → corto (120s)
```

A.12 trusteando/adapters/ — opcional

Adaptadores que permiten a organismos con bases de datos propias exponer sus datos con la misma interfaz que las carpetas del protocolo. El organismo publica la función, el parser la llama con rutas del protocolo y recibe valores como si fueran carpetas.

```
trusteando/adapters/
  [nombre_funcion]/
    name
    description
    input_schema          ← campos del protocolo que necesita
    output_schema         ← campos que devuelve
    version
    cache/ttl
```

A.13 Campos obligatorios, recomendados y opcionales

OBLIGATORIO — sin estos campos el nodo no es válido para el protocolo:

- trusteando/identity/public_key
- trusteando/identity/hash_migracion
- trusteando/identity/hash_disputa
- trusteando/identity/entity_type
- trusteando/identity/location/lat y lon — si existe location/

RECOMENDADO — opcional pero mejora la interoperabilidad:

- trusteando/identity/scope
- trusteando/identity/root_certificate — si existe reconocimiento formal
- trusteando/identity/contact/ — para negocios
- trusteando/identity/location/ — para negocios físicos
- trusteando/status/ — para negocios con horario
- trusteando/notifications/ — para nodos con cambios frecuentes

OPCIONAL — libre, pero si se implementa debe seguir el esquema estándar:

- trusteando/identity/classification/
- trusteando/status/queue/
- trusteando/status/occupancy/
- trusteando/routing/
- trusteando/adapters/
- trusteando/features/
- trusteando/payments/
- trusteando/messaging/
- trusteando/media/
- trusteando/hooks/

Antes de crear un campo nuevo, verifica si algún campo estándar ya cumple esa función. Usar campos estándar garantiza que parsers, agregadores, y aplicaciones de terceros puedan interpretar tu nodo sin configuración adicional. Los campos no estándar son ignorados por implementaciones que no los conocen — lo que es correcto — pero se pierde interoperabilidad. Si creas un campo no estándar que aporta valor diferencial, decláralo bajo trusteando/extensions/ con una justificación.

A.14 Convención de booleanos por presencia

En el protocolo, la presencia de una carpeta indica verdadero — su ausencia indica falso. No hay campos con valor bool explícito. Esto simplifica la verificación — un parser comprueba si la carpeta existe, no si el valor es true o false. Y simplifica la publicación — el negocio crea o elimina carpetas, no edita valores.

```
trusteando/payments/accepted/
  cash/           ← existe → acepta efectivo
  card/           ← existe → acepta tarjeta
  google_pay/     ← existe → acepta Google Pay
                  ← bizum/ no existe → no acepta bizum

trusteando/features/
  parking/        ← existe → tiene parking
  wifi/           ← existe → tiene wifi
```

```
                    ← pool/ no existe → no tiene piscina
```

A.15 trusteando/features/ — opcional

Características verificables del negocio — hechos estructurados que parsers y agregadores pueden usar para filtrar y comparar. No es marketing — es información interoperable. Algunos campos pueden tener un certified_by si un organismo los verifica.

```
trusteando/features/
  parking/                ← bool por presencia
    type/                 ← free/ | paid/ | valet/
    spaces                ← número de plazas
  wifi/                   ← bool por presencia
    speed_mbps
  accessibility/          ← bool por presencia
    wheelchair/           ← bool por presencia
    elevator/             ← bool por presencia
    certified_by          ← organismo verificador opcional
  pets/                   ← bool por presencia
    conditions            ← texto libre
  pool/                   ← bool por presencia
    type/                 ← indoor/ | outdoor/ | both/
  checkin/
    from                  ← HH:MM
    until                 ← HH:MM
  checkout/
    until                 ← HH:MM
  restaurant/             ← bool por presencia
    cuisine               ← tipo de cocina
  smoking/                ← bool por presencia — permite fumar
  air_conditioning/       ← bool por presencia
  heating/                ← bool por presencia
```

A.16 trusteando/payments/ — opcional

Métodos de pago aceptados y hook al proveedor de pagos. El protocolo no implementa pagos — define dónde se conectan. Cambiar de proveedor es modificar un solo campo.

```
trusteando/payments/
  accepted/               ← métodos aceptados — booleanos por
presencia
    cash/
    card/
    contactless/
    bizum/
    google_pay/
    apple_pay/
    invoice/              ← acepta factura
  agent/                  ← hook al proveedor de pagos
    provider              ← URL del nodo del proveedor
    endpoint              ← donde se inicia la transacción
    externals/
      stripe
      redsys
      paypal
```

A.17 trusteando/messaging/ — opcional

Hook al proveedor de mensajería. El protocolo no implementa mensajería — define dónde se conecta. Se recomienda priorizar protocolos abiertos como Matrix sobre soluciones propietarias.

```
trusteando/messaging/
  agent/
    provider              ← URL del nodo del proveedor
    externals/
      whatsapp
      telegram
      matrix              ← protocolo abierto recomendado
      email               ← fallback universal
```

A.18 trusteando/hooks/ — opcional

Punto de conexión general para servicios externos. Los hooks siguen un principio deliberado: el negocio nunca debe estar atado a un proveedor específico por razones técnicas. El protocolo define los puntos de conexión — el mercado decide quién los implementa mejor. Cambiar de proveedor debe ser siempre una decisión de negocio, nunca una migración técnica.

```
trusteando/hooks/
  payments/agent/provider     ← proveedor de pagos
  messaging/agent/provider    ← proveedor de mensajería
  booking/agent/provider      ← proveedor de reservas
  delivery/agent/provider     ← proveedor de entregas
  analytics/agent/provider    ← proveedor de analíticas
```

A.19 trusteando/media/ — opcional

Imágenes y contenido multimedia verificable. Los hashes garantizan que el contenido no ha sido alterado. El almacenamiento real puede estar en cualquier CDN — el protocolo solo registra el hash y la referencia.

```
trusteando/media/
  photos/
    [photo_id]/
      hash              ← SHA-256 del fichero
      url               ← donde está almacenada la imagen
      caption           ← descripción
      taken_at          ← fecha
      certified_by      ← quien verificó que es real (opcional)
  logo/
    hash
    url
```

A.20 trusteando/rescue/ — opcional pero crítico

La carpeta de resiliencia declara dónde están las copias operativas del nodo si el servidor principal cae. Tiene una propiedad especial respecto al caché: es la única carpeta del protocolo con prioridad de caché obligatoria.

La paradoja de rescue/ es que si solo se lee cuando el servidor cae, ya es demasiado tarde para leerla. Por eso todo parser que visite un nodo debe cachear rescue/ en su primera visita y renovarla periódicamente — independientemente del TTL del resto del árbol. Un parser que no tiene rescue/ en caché no puede ofrecer resiliencia ante caídas.

```
trusteando/rescue/
  cache/
    ttl                 ← 604800s (7 días) por defecto
    priority            ← max — se cachea antes que cualquier otra
carpeta
    strategy            ← always — nunca se omite aunque el resto no
se cachee
  mirrors/              ← copias del nodo en otros servidores
    [mirror_id]/
      url               ← URL del mirror
      last_synced       ← timestamp de la última sincronización
      operator          ← quién opera el mirror
      verified/         ← bool por presencia — mirror verificado
  fallback/
    ttl                 ← cuánto tiempo es válida la copia cacheada
  backup/
    frequency           ← cada cuánto se hace backup
    last_backup         ← timestamp del último backup
    externals/
      github            ← repo donde se publica el backup
      ipfs              ← si usa IPFS para distribución
  emergency/
    contact/
      primary           ← contacto principal
      secondary         ← contacto alternativo
      protocol          ← URL al documento de procedimiento
    escalation/
      level_1           ← primer escalado si primary no responde
      level_2           ← segundo escalado
      root              ← escalar al root como último recurso
    status_page         ← URL donde se publica el estado del nodo
    hook/               ← comodín para sistemas de gestión de
emergencias
      provider          ← URL del sistema externo
      endpoint          ← donde se notifica el problema
      severity_levels/  ← niveles de severidad que maneja
      externals/        ← si usa sistemas conocidos
        pagerduty
        opsgenie
        victorops
```

Para casos de uso con requisitos de disponibilidad críticos, el hook emergency/hook/ permite conectar el protocolo con cualquier sistema de gestión de incidentes externo. El protocolo no especifica ese sistema — solo define el punto de conexión. Las extensiones necesarias para esos casos de uso se declaran bajo trusteando/extensions/ siguiendo el proceso estándar.

El orden de caché recomendado para cualquier parser:

```
1. rescue/         ← SIEMPRE — primera visita y renovación periódica
2. identity/       ← alta prioridad — cambia poco
3. notifications/  ← para detectar cambios posteriores
4. registry/       ← normal
5. status/         ← corto o no_cache
6. resto           ← según su propio cache/ttl declarado
```

A.21 trusteando/safety/ — opcional, recomendado para locales públicos

Información de seguridad del establecimiento. Tiene dos capas complementarias: información básica local — disponible desde el principio, publicada por el propio establecimiento — y referencias a agentes externos con autoridad real sobre la seguridad del local.

La presencia de fully_implemented/ junto a un agente indica que ese agente tiene una implementación completa y específica para este establecimiento en su propio nodo. Su ausencia indica que la información local puede ser la única disponible.

```
trusteando/safety/

  ← Información básica local
  emergency_exits/
    [exit_id]/
      location          ← descripción o coordenadas
      floor
      accessible/       ← bool por presencia
  assembly_point        ← punto de encuentro exterior
  capacity/
    max                 ← aforo máximo legal
    certified_by
  defibrillator/        ← bool por presencia
    location
  first_aid/            ← bool por presencia
    location
  evacuation/
    plan_url

  ← Referencias a agentes con autoridad real
  authorities/
    fire/
      node              ← URL nodo bomberos con entrada específica
      fully_implemented/ ← bool por presencia — plan completo
verificable
      externals/phone   ← fallback mientras no hay nodo completo
    police/
      node
      fully_implemented/
      externals/phone
    medical/
      node
      fully_implemented/
      externals/phone
    food_safety/
      node
      fully_implemented/
    building/
      node              ← ayuntamiento — licencias y normativa
edilicia
      fully_implemented/
    insurance/
      node              ← aseguradora — póliza activa
      fully_implemented/
```

La seguridad de un local no es lo que el local dice sobre sí mismo — es la red de organismos con autoridad que han implementado algo específico sobre él. La información local bajo safety/ es siempre útil como punto de partida y como redundancia. La presencia de fully_implemented/ junto a un agente indica que la fuente autoritativa está en el nodo externo y que el trabajo previo está completado y verificable.

A.22 trusteando/registry/compromised/ — opcional

Lista de claves públicas o hashes públicos que el emisor declara como comprometidos o inválidos. Permite la revocación proactiva por parte del emisor sin esperar a que el propio sujeto inicie una migración.

El caso de uso principal es la protección del ecosistema cuando el emisor tiene evidencia de que una clave ha sido vulnerada antes de que el sujeto pueda actuar: una universidad que detecta que las credenciales de un miembro han sido robadas puede publicar el hash_publico comprometido en esta carpeta de forma inmediata. Cualquier verificador que consulte registry/compromised/ sabe que esa clave no debe aceptarse aunque la firma sea criptográficamente válida.

```
trusteando/registry/compromised/
    {hash_publico_comprometido}/    ← bool por presencia: esta clave no debe aceptarse
        since/2026-03-18T10:30:00Z   ← fecha de deteccion del compromiso
```

Esta carpeta es complementaria al mecanismo de migración del sujeto (sección 4.7) y al state brokenado de la gramática (A.22). Juntos cubren los tres vectores de revocación: el sujeto migra voluntariamente, el emisor revoca proactivamente, y la red observa y declara el state brokenado mediante quórum.

El protocolo define una jerarquía de resolución de disputas análoga al recurso de alzada en derecho administrativo. Las disputas se resuelven al nivel más bajo posible. Elevar una disputa tiene un coste creciente para desincentivar el uso frívolo.

B.1 Jerarquía de resolución

```
Nivel 1 — el superior inmediato
  Juan tiene disputa con la universidad
  → la universidad resuelve usando hash_disputa de Juan
  Coste: bajo

Nivel 2 — recurso de alzada
  Juan no acepta la resolución de la universidad
  → eleva al nodo superior de la universidad
  → ese nodo resuelve con su propia autoridad
  Coste: medio

...

Nivel N — el root como tribunal de última instancia
  Solo para disputas que han agotado niveles inferiores
  O para disputas entre nodos sin superior común
  Coste: elevado
```

B.2 El coste de disputa

Cada nivel tiene un coste creciente — económico y de reputación. Elevar una disputa al root sin haber pasado por los niveles inferiores no está permitido salvo casos excepcionales tasados. Perder una disputa que se elevó innecesariamente tiene consecuencias para el nodo que la elevó. El modelo es análogo a las tasas judiciales: existen para que los recursos sean proporcionales a la gravedad del asunto.

B.3 El rol de hash_disputa

El hash_disputa — derivado de clave_emergencia_2, nunca revelado en migraciones normales — es la prueba de identidad ante el árbitro. En una disputa el nodo demuestra que conoce la clave que produce ese hash sin revelarla. El árbitro verifica la prueba y resuelve. Si la disputa sube de nivel, el hash_disputa acompaña el expediente — es la cadena de custodia criptográfica de la identidad del disputante.

A.23 Gramática de rutas: gramática definitiva

Los segmentos de una ruta son de tres tipos. El parser los distingue por la presencia de corchetes y por la presencia de espacio dentro de ellos. Los únicos caracteres reservados del protocolo son /, [y].

Tipo 1 — Identificador canónico (sin corchetes)

`hospital-la-paz` `restaurante-casa-robles` `universidad-malaga`

Nodo del grafo con identidad propia y autoridad sobre su espacio. La convención de nomenclatura es `tipo-nombre`: el tipo forma parte del identificador, lo hace canónico y autoexplicativo fuera de contexto. Dos nodos consecutivos separados por / implican relación jerárquica con transferencia de autoridad.

Tipo 2 — Relación (corchetes, sin espacio interior)

`[en]` `[accredited-by]` `[funded-by]` `[agreement-with]`

Operador semántico que conecta nodos. Expresa una relación contextual sin transferencia de autoridad: el nodo que precede al operador no certifica al que le sigue. El vocabulario es abierto — cualquier término en lenguaje natural es válido.

Tipo 3 — Propiedad con valor (corchetes, con espacio interior)

`[camas 150]` `[fundado 1964]` `[estrellas 2]` `[temperatura 23.5]`

Atributo del nodo precedente. El valor es un literal inmutable — no es una entidad navegable ni tiene URL propia. El espacio dentro de los corchetes es el marcador semántico que distingue este tipo del anterior.

Rutas completas:

```
hospitales/[en]/madrid/hospital-la-paz/[camas 150][fundado 1964]
restaurantes/[en]/sevilla/restaurante-casa-robles/[estrellas 2][fundado 1952]
obras/[funded-by]/ue/obra-autovia-a7/[km 45][budget_euros 4500000]
sensor-42/[installed-in]/edificio-a/[planta 3][temp_celsius 85]
```

El parser:

```
def parse_segment(s):
    if s.startswith('[') and s.endswith(']'):
        inner = s[1:-1]
        if ' ' in inner:
            key, value = inner.split(' ', 1)
            return {'type': 'property', 'key': key, 'value': value}
        return {'type': 'relation', 'value': inner}
    return {'type': 'node', 'value': s}

def parse_path(path):
    return [parse_segment(s) for s in path.strip('/').split('/')]
```

Esta gramática no reemplaza la convención since/until para hechos temporales ni la carpeta externals/ para vínculos sin control. Es una extensión ortogonal que permite expresar

relaciones y propiedades directamente en la ruta de forma autodocumentada, sin añadir carga al protocolo base.

Convenciones semánticas reservadas

Sobre la gramática base se definen las siguientes convenciones semánticas. No añaden nuevos tipos de segmento ni nuevos caracteres reservados — son valores convencionales dentro del tipo 3 que el protocolo interpreta de forma estandarizada.

Relaciones recíprocas: clave `mutua`

Cuando la clave de una propiedad es `mutua`, el valor expresa el tipo de relación y la contraparte. La relación es recíproca: para considerarse verificada, la contraparte debe publicar una declaración equivalente. Hay dos formas:

`pedro/[mutua amistad juan]/` ← bilateral: pedro declara amistad mutua con juan
`juan/[mutua amistad pedro]/` ← juan confirma: relación verificada bilateralmente

Para relaciones entre más de dos nodos se usa el valor reservado `among-members` en un nodo colectivo. El nodo declara el tipo de relación que existe entre todos sus miembros:

```
grupo-amigos-madrid/[mutua amistad among-members]/  ← relación de amistad mutua entre todos los miembros
grupo-amigos-madrid/[miembro pedro]/
grupo-amigos-madrid/[miembro juan]/
grupo-amigos-madrid/[miembro ana]/
```

Distinción crítica: atributo vs subordinación

La elección entre tipo 1 y tipo 3 para expresar la relación de un nodo con sus miembros tiene consecuencias semánticas precisas:

`grupo/miembro/juan/` ← juan está subordinado al grupo. El grupo tiene autoridad sobre él.
`grupo/[miembro juan]/` ← juan es un atributo del grupo. No hay subordinación ni autoridad.

Para grupos de iguales — amigos, socios, colaboradores — siempre se usa la forma de atributo. La forma de subordinación se reserva para relaciones jerárquicas reales donde el nodo superior tiene autoridad sobre el inferior.

Vocabulario reservado de estados

El protocolo reserva tres términos como estados canónicos de un nodo. Son extensibles en versiones futuras pero no redefinibles por implementaciones particulares:

`trusteando` ← confianza plena, verificado con pruebas sólidas
`verifiado` ← cumple mínimos, aprobado básico
`brokenado` ← nodo comprometido parcialmente: su clave local ha sido vulnerada pero su identidad sigue siendo válida en el quórum restante de roots. No equivale a inválido.

Convención [indicates] para afirmaciones con fuente

Para expresar que un estado o propiedad es afirmado por una entidad externa se usa el operador relacional `[indicates]` seguido de la fuente. Esto permite atribuir cada afirmación a su origen, tener múltiples fuentes sobre el mismo nodo, y desvincular el protocolo de infraestructura específica. El protocolo no designa fuentes oficiales: cualquier nodo puede actuar como fuente indicando estados sobre otros nodos.

```
restaurante-casapepe/[state trusteando]/[indicates]/[fuente michelin.es]/
root-23/[state brokenado]/[indicates]/[observador root-45]/[observador root-78]/
restaurante-casapepe/[state trusteando]/[indicates]/[fuente michelin.es]/[fuente tripadvisor.com]/
```

Azúcar sintáctico: [X by Y] para estados

Para facilitar la escritura humana, el parser puede transformar la forma `[X by Y]` en `[estado X][contexto Y]` si y solo si X es un término del vocabulario reservado de estados (actualmente trusteando, verifiado, brokenado). Esta condición es esencial: garantiza que parsers que no implementen el azúcar encuentren una degradación predecible — ven X como parte reconocible del vocabulario — en lugar de una interpretación silenciosamente incorrecta.

```
[brokenado by reputacion]   →  [state brokenado][context reputation]
[trusteando by node-state]  →  [state trusteando][context node-state]
[reserva by ventana]         →  propiedad ordinaria (reserva no es estado
reservado)
```

Esta convención no introduce nuevas palabras reservadas en la gramática, solo en el vocabulario semántico. La gramática de tres tipos permanece intacta. `by` es texto libre en cualquier contexto salvo cuando precede a un valor y está precedido por un estado reservado.

Nota: La gramática de Trusteando puede entenderse por analogía con la programación orientada a objetos. Las carpetas son objetos. Las subcarpetas son métodos (acciones) o colecciones. Los corchetes son propiedades o valores. Y como establece la sección 2.10, una vez firmados, los valores entre corchetes son inmutables — exactamente igual que los campos de un objeto inmutable en un lenguaje funcional. La diferencia fundamental es que en Trusteando no hay encapsulación: todo es público por defecto salvo lo declarado bajo private/. El grafo es un espacio de objetos verificables, no de estados privados.

A.24 Convención plan/execution para procesos activos

La convención since/until describe hechos consumados: períodos con inicio y fin ya conocidos. No es suficiente para representar procesos activos donde existe una intención planificada y una ejecución que puede diferir de ella. Para estos casos el protocolo define la convención plan/execution.

Cualquier acción, orden, tarea o proceso puede contener dos subcarpetas:

```
accion/
├── plan/        ← intención, firmado por quien ordena o planifica
│   └── [libre]   ← start/, end/, resources/, priority/... lo que el dominio
necesite
└── execution/  ← realidad, firmado por quien ejecuta
    └── [libre]   ← start/, end/, reports/, incidents/... lo que el dominio
necesite
```

El contenido de cada subcarpeta es libre: el protocolo no prescribe qué carpetas van dentro. Cada dominio define el vocabulario que necesita. Esta libertad no sacrifica interoperabilidad — la distinción entre intención y realidad es universal; el vocabulario interno es específico del contexto.

La semántica de booleanos por presencia se aplica directamente: la ausencia de execution/end/ indica que el proceso está en curso. Su presencia indica que ha concluido. No hay campo de estado explícito — el estado se lee en la estructura.

El control de acceso sigue la regla general del protocolo: quien controla la URL bajo la que publica tiene autoridad sobre ese contenido. plan/ lo firma quien ordena; execution/ lo firma quien ejecuta. Un verificador externo puede comparar ambas subcarpetas y derivar el estado real del proceso — retraso, cumplimiento, desviación — sin necesidad de consultar a ninguna de las partes.

Ejemplos de aplicación:

— Emergencias: orden-001/plan/end/ es el deadline; orden-001/execution/end/ es cuándo se completó.
— Construcción: cimentacion/plan/end/ es la fecha prevista; cimentacion/execution/porcentaje/ es el avance real.
— Trámite administrativo: solicitud/plan/resolucion/ es el plazo legal; solicitud/execution/resolucion/ es la fecha real.
— Software: sprint/plan/story-points/ es la estimación; sprint/execution/completados/ es lo entregado.

Apéndice C — API mínima del protocolo

La API define cómo se interactúa con un nodo. Es independiente de la implementación — un nodo puede ser un servidor web estático, una API REST, o un microservicio, mientras exponga estos métodos con la misma semántica. La API es una conveniencia para facilitar la interoperabilidad — lo que no esté aquí puede implementarse libremente sin romper el protocolo.

C.1 Verificación

```
verify(url, folder)
  → ¿está este nodo en esta carpeta ahora?
  → devuelve: { valid: bool, timestamp: ISO8601, issuer: url }

getIdentity(url)
  → obtener trusteando_identity/ de un nodo
  → devuelve: { public_key, hash_migracion, hash_disputa,
                entity_type, scope, root_certificate }
```

C.2 Ciclo de vida

```
create(url, entity_type, scope)
  → crear un nuevo nodo en el sistema
  → genera claves, publica trusteando_identity/

certify(subject_url, folder)
  → añadir un nodo a una carpeta del registro
  → equivale a emitir una credencial

revoke(subject_url, folder)
  → eliminar un nodo de una carpeta del registro
  → equivale a revocar una credencial específica
```

C.3 Migración

```
migrate(new_superior_url, emergency_key_1)
  → solicitar migración a un nuevo superior
  → el nodo revela emergency_key_1 al nuevo superior

adoptNode(subject_url, emergency_key_1)
  → el superior acepta la migración
```

```
  → verifica emergency_key_1 contra hash_migracion publicado
  → publica old_identities/ con cadena completa
  → aprueba ante el root
```

C.4 Privacidad contextual

```
requestReveal(subject_url, field, context)
  → el receptor solicita acceso a un campo protegido
  → field: ruta de la carpeta private/
  → context: propósito declarado de la solicitud
  → devuelve: { request_id, status: pending }

grantReveal(request_id, disclosure_level)
  → el sujeto concede acceso al campo solicitado
  → disclosure_level: RESULT_ONLY | ISSUER_ONLY |
                      CONTENT | FULL_CHAIN
  → transferable: bool  ← si el receptor puede demostrarlo a terceros

denyReveal(request_id)
  → el sujeto deniega la solicitud
```

Apéndice D — Capa de negocio — transactions/trusteando/

El protocolo Trusteando es general — sirve para identidad, certificaciones, y cualquier hecho verificable. La capa de negocio es una extensión opcional que estandariza cómo se expresan transacciones y reputación verificables. Cualquier nodo que quiera participar en el ecosistema de transacciones verificables adopta la convención transactions/trusteando/.

La convención es voluntaria pero estratégicamente importante: cualquier sistema que quiera indexar, agregar, o verificar transacciones y reputación de forma interoperable tiene que hablar este lenguaje. La carpeta es la declaración de participación — no hace falta registro central ni API especial.

D.1 Estructura de transactions/trusteando/

```
casapepe.es/transactions/trusteando/

  visits/                              ← visitas verificadas
    since/YYYY-MM-DD/[visitor_hash]

  reviews/                             ← colección de reseñas
    review/                            ← objeto reseña (datos
primarios)
      set_id                           ← identificador único
      type                             ← entero enumerado (ver D.2)
      content_hash                     ← SHA-256 del texto
      rating                           ← valoración numérica
      visit_ref                        ← referencia a visita (si type
>= 2)
```

```
      payment_ref                       ← referencia a pago (si type >=
3)
      private/             ← campos privados opcionales
    calculated/                         ← índices derivados
automáticamente
      verified_visit/                   ← reviews con type >= 2
      verified_paid/                    ← reviews con type >= 3
      verified_identity/                ← reviews con type >= 1
      by_rating/                        ← índice por valoración

  offers/
    [offer_id]/
      terms
      valid/since/until/
      eligibility/

  reputation/
    calculated/
      score                             ← agregado calculado
      total_reviews                     ← conteo de set_id únicos
      total_verified_visits             ← conteo type >= 2
      total_verified_paid               ← conteo type >= 3
```

D.2 Tipo enumerado de reseña

El campo type es un entero enumerado donde el orden refleja el nivel de verificación — a mayor número, mayor garantía criptográfica:

type	nombre	garantía
0	unverified	sin verificación — cualquiera puede opinar
1	verified_identity	identidad del autor verificada
2	verified_visit	presencia física verificada
3	verified_paid	compra verificada por TPV o token de pago
4	verified_visit_and_paid	presencia física + compra verificadas

El orden importa para consultas: reviews con type >= 2 devuelve solo reseñas con presencia física verificada. Contar reseñas únicas es siempre contar set_id distintos en reviews/review/ — los índices en calculated/ son vistas derivadas, no duplicados.

D.3 La carpeta calculated/

Todo lo que está bajo calculated/ es derivado automáticamente de los datos primarios y verificable independientemente por cualquier tercero. Nunca se escribe a mano. El protocolo define dos funciones para gestionar campos calculados:

```
updateIndex(entity_url, collection)
  → recalcula los índices de una colección
  → se llama tras cada create/update/delete de un objeto
  → idempotente — ejecutarlo dos veces da el mismo resultado

checkIndex(entity_url, collection)
  → verifica que los índices coinciden con los datos primarios
  → devuelve: { valid: bool, mismatches: [...] }
```

```
  → cualquier tercero puede ejecutarlo — auditoría pública autónoma
```

D.4 La carpeta routing/

Declara explícitamente a quién envía datos una entidad y para qué. Es transparencia activa — cualquier tercero puede ver exactamente qué procesadores tienen autorización. Compatible con el registro de actividades de tratamiento del GDPR.

```
trusteando/routing/
  processors/           ← entidades autorizadas a procesar mis datos
    confidencenode.org  ← el root — verificaciones
    analytics.es        ← procesador de analíticas autorizado
  subscribers/          ← entidades que reciben notificaciones de
eventos
    malaga.org          ← agregador regional suscrito
  webhooks/             ← endpoints que reciben eventos
automáticamente
```

D.5 Semántica de los prefijos reservados

carpeta	semántica
`externals/`	enlace a algo no controlado por el nodo
`calculated/`	derivado automáticamente, verificable con checkIndex()
`private/`	existe pero no expuesto por defecto — grantReveal() para acceso
`routing/`	procesadores y suscriptores autorizados a recibir datos
`fully_implemented/`	bool por presencia — el nodo externo referenciado tiene implementación completa y específica. Su ausencia indica implementación parcial o en progreso — la información local puede ser la única disponible.

D.6 La propiedad central

Una reseña bajo transactions/trusteando/reviews/review/ no es una opinión — es una arista del grafo que conecta un nodo verificado con una visita o compra verificada. El contenido puede ser subjetivo pero la presencia es objetiva. Cualquier verificador puede comprobarlo consultando URLs públicas.

Esta propiedad destruye las reseñas falsas por construcción, no por moderación. Y la auditoría es pública y autónoma — cualquiera puede ejecutar checkIndex() y confirmar que los índices de reputación son consistentes con los datos primarios.

D.7 Valor estratégico del nombre

La convención transactions/trusteando/ asocia el nombre Trusteando con el estándar de transacciones verificables del protocolo. Cualquier implementación que adopte esta convención refuerza ese vínculo. El dominio trusteando.app es la implementación de referencia — el punto de entrada natural para cualquiera que busque qué significa esta carpeta o cómo implementarla.

Apéndice E — Diccionario de intencionalidad — inspiración en mecanismos web

Trusteando toma inspiración de mecanismos web establecidos. El diccionario siguiente ayuda a entender la intencionalidad de cada carpeta — no implica equivalencia técnica ni compatibilidad directa. Donde la web resolvió problemas similares, Trusteando añade firma criptográfica, semántica de identidad, y verificabilidad pública.

Trusteando	Inspirado en	Diferencia clave
`trusteando/redirect/`	HTTP 301/302	Firmado, forma parte del grafo de identidad, con fecha y cadena histórica
`trusteando/notifications/`	HTTP headers / ETag	Declarativo en carpeta, verificable, sin negociación de cabeceras
`notifications/changelog/`	RSS / Atom	Sin servidor de feed, pull directo sobre carpetas públicas
`trusteando/routing/`	robots.txt / CORS	Con semántica de autorización firmada y compatible con GDPR
`trusteando/registry/`	sitemap.xml	Con firmas criptográficas y semántica de credencial
`calculated/`	Cache / ETag	Auditable públicamente con checkIndex() — no solo para rendimiento
`externals/`	<a href> externo	Declara explícitamente falta de control sobre el destino
`routing/processors/`	GDPR registro tratamiento	Verificable criptográficamente, no solo declarativo
`private/`	— (sin equivalente)	Privacidad contextual con divulgación selectiva — no existe en la web

La carpeta trusteando/redirect/ merece mención especial. A diferencia de un HTTP 301, un redirect en Trusteando es una declaración firmada de migración de identidad — con fecha, con referencia a la clave de emergencia usada, y formando parte del grafo histórico del nodo. No es solo una instrucción técnica de enrutamiento sino un hecho verificable con las mismas garantías que cualquier otra credencial del protocolo.

```
trusteando/redirect/
  target          ← URL de destino (nueva identidad)
  permanent       ← bool — migración definitiva o temporal
  since           ← fecha de la migración
  migration_ref   ← referencia al proceso de migración en
old_identities/
```

Apéndice F — Identidad del autor, cadena de versiones y modelo de gobernanza

Este apéndice aplica el propio protocolo para establecer la identidad del autor, la cadena de autenticidad entre versiones del whitepaper, y el modelo de gobernanza inicial. Es a la vez una especificación y una demostración: el autor usa Trusteando para demostrar que es el autor de Trusteando.

F.1 Identidad canónica del autor

La identidad del autor usa la modalidad autónoma definida en la sección 4.10. El identificador canónico h1 incorpora directamente la gramática del protocolo, lo que lo hace semánticamente verificable por cualquiera que entienda la especificación:

```
h1 = hash("ConfidenceNode0/[author-of Trusteando Protocol]")
h2 = hash(h1 + secret_v02)         ← hash público de esta versión
h3 = hash(h1 + secret_disputes)    ← clave de disputas, independiente
```

h1 es permanente y público: cualquiera puede calcularlo aplicando SHA-256 a la cadena literal. h2 cambia en cada versión y solo el autor puede generarlo porque requiere secret_v02, que nunca se publica. h3 es independiente de h2 — comprometer uno no compromete el otro.

La gestión de los secretos no requiere que ningún humano los memorice. El wallet de referencia (previsto en v0.2) los genera, los custodia en el enclave seguro del dispositivo, y distribuye copias cifradas a wallets colaboradores de confianza usando el propio protocolo. El autor otorga el rol de custodia publicando bajo su propia URL — nadie puede autoproclamarse custodio:

```
author/s-w-c/
├── [w-c-1 secret-wallet-collaborator]/
│   ├── since/2026-03-18/             ← inicio de la custodia
│   └── [state trusteando]/          ← activo y verificado
├── [w-c-2 secret-wallet-collaborator]/
│   ├── since/2026-03-18/
│   └── [state trusteando]/
├── [w-c-3 secret-wallet-collaborator]/
│   ├── since/2026-03-18/
│   └──                              ← vacío hasta que esté verificado
└── [quorum 2]/                      ← mínimo para reconstruir el
secreto
```

Si una custodia es revocada o comprometida, el autor elimina o actualiza la entrada bajo s-w-c/ y el estado pasa a brokenado. El ciclo de vida completo — otorgamiento, activación, revocación — usa exclusivamente la gramática estándar del protocolo. No hay APIs especiales ni tablas de permisos externas.

F.2 Cadena de versiones

Cada versión del whitepaper publica su h2 y un migration_ref que la vincula con la versión anterior. Solo quien conocía el secreto de la versión anterior puede generar un migration_ref válido. La cadena es unidireccional: la versión N acredita a la versión N+1, nunca al revés.

```
confidencenode.org/protocolos/trusteando/
├── v0.1/
│   ├── h1/                       ← identidad canónica del autor
│   └── h2/                       ← hash público v0.1
├── v0.2/
│   ├── h1/                       ← mismo h1, permanente
```

```
│   ├── h2/                        ← hash público v0.2
│   ├── migration_ref/             ← hash(h2_v01 + h2_v02)
│   └── [acredited-by]/v0.1/       ← solo la versión anterior acredita
└── v0.N/  ← cada versión futura extiende la cadena del mismo modo
```

Un lector que encuentre el whitepaper en cualquier repositorio puede verificar su autenticidad consultando la cadena en confidencenode.org — no necesita confiar en el repositorio. Un documento que afirme ser v0.3 sin tener un migration_ref válido que lo vincule a v0.2 no es auténtico.

F.3 Modelo de gobernanza inicial y transición al policentrismo

El autor no activa su rol de root hasta que se cumplen simultáneamente un conjunto de condiciones de lanzamiento — condiciones expresadas en el propio protocolo usando calculate/maximum. Esta decisión es deliberada: no asumir responsabilidad institucional antes de que el sistema tenga las herramientas y los avales necesarios. La acreditación de nuevos roots usa la convención [agent role]: clave es el identificador del agente, valor es su rol en el ecosistema.

```
author/
├── h1/
├── [author-of Trusteando Protocol]/
├── [confidencenode0 root-founding-author]/confidencenode0/
│   ├── [state verifiado]/          ← identidad establecida, root no
activo aún
│   └── from/calculate/maximum/      ← todas las condiciones deben
cumplirse
│       ├── [time secure-wallet-is-ready]/
│       ├──
group-of-legal-experts-selected-for-launch/execution/[state ready]/
│       ├──
group-of-security-experts-selected-for-launch/execution/[state ready]/
│       └──
group-of-computer-science-experts-selected-for-launch/execution/[state
ready]/
├── [spain root-national-agent]/spain-root-national-agent/
├── [france root-national-agent]/france-root-national-agent/
├── [eu root-supranational-agent]/eu-root-agent/
└── [quorum 3]/               ← mínimo de roots para operaciones
sensibles
```

La convención [agent role] es extensible sin límite. Cualquier entidad puede ser acreditada con cualquier rol. Los roles actuales son root-founding-author, root-national-agent, y root-supranational-agent, pero el protocolo no restringe el vocabulario. Cualquier verificador puede descubrir todos los roots activos buscando nodos con el sufijo -root- en su rol — sin consultar ningún registro central.

La transición al policentrismo es irreversible por diseño: una vez que los agentes nacionales están acreditados y operando con quórum propio, el sistema no depende de la firma del autor para funcionar. El autor no puede recuperar control unilateral. La acreditación es un acto público, firmado, y permanente en el grafo.

Cada grupo de expertos tiene su propia estructura interna. El autor nombra al grupo asignándole el rol approve-launch-conditions publicando bajo su propia URL. El grupo nombra a sus miembros y controla cuándo declara su aprobación publicando bajo la suya. Ninguna de las dos partes puede falsificar la acción de la otra:

```
author/[group-of-computer-science-experts-selected-for-launch approve-
launch-conditions]/
    ← el autor nombra al grupo y le asigna el rol
```

```
group-of-computer-science-experts-selected-for-launch/
├── [expert-1 is-member]/    ← el grupo nombra a sus miembros
├── [expert-2 is-member]/
├── [expert-3 is-member]/
├── [quorum 3]/              ← mínimo para la aprobación del grupo
├── plan/[state approved]/        ← el grupo aprueba el plan
└── execution/[state ready]/    ← el grupo declara condiciones
satisfechas
```

La separación de autoridades es total: el autor controla quién es el experto, el experto controla cuándo aprueba. Cuando los tres grupos hayan publicado [state ready] bajo sus propias URLs, y el wallet esté listo, calculate/maximum evaluará todas las condiciones como satisfechas y el root quedará activo automáticamente — sin ningún acto adicional del autor.

F.4 Ejemplo ilustrativo de estructura interna de un agente nacional (no normativo)

Las decisiones de gobernanza interna de cada agente nacional son soberanas. El protocolo no impone ningún modelo. El siguiente ejemplo es puramente ilustrativo — una posible estructura que un estado podría adoptar como punto de partida, adaptándola libremente a su marco jurídico e institucional.

```
spain-root-national-agent/
├── [acredited-by]/author/          ← acreditado por el autor fundador
├── [quorum 3]/                    ← quórum interno para operaciones
sensibles
├── [sepe root-domain-agent]/sepe-root/   ← agente de dominio: empleo
├── [mec root-domain-agent]/mec-root/    ← agente de dominio:
educación
├── [aeat root-domain-agent]/aeat-root/  ← agente de dominio: hacienda
├── regions/
│   ├── [andalucia root-regional-agent]/andalucia-root/
│   └── [catalunia root-regional-agent]/catalunia-root/
└── [since 2027-01-01]/               ← fecha de activación del agente
nacional
```

En este ejemplo, el agente nacional español delega dominios sectoriales a ministerios y dominios territoriales a comunidades autónomas, cada una con su propio nodo y rol. La jerarquía es la que el estado decida — el protocolo solo provee la gramática. Un verificador externo que consulte spain-root-national-agent puede descubrir automáticamente todos los agentes subordinados sin ningún directorio central.

F.5 Ejemplo ilustrativo: suscripción de España al protocolo mediante publicación en el BOE (no normativo)

El Boletín Oficial del Estado es el mecanismo de publicidad jurídica oficial de España. Cualquier acto publicado en el BOE tiene validez legal en el ordenamiento español. Su uso como ancla de suscripción al protocolo resuelve en un solo paso el problema del arranque institucional: la publicación en el BOE es la declaración oficial de adhesión y la prueba pública de la clave del agente nacional, verificable por cualquier ciudadano, empresa o administración.

El flujo ilustrativo sería el siguiente. El Gobierno aprueba la adhesión al protocolo Trusteando mediante orden ministerial. La orden se publica en el BOE con el contenido mínimo necesario: la URL del nodo raíz nacional, la clave pública del agente, y el hash público que acredita la identidad. A partir de ese momento cualquier verificador puede consultar el BOE y el nodo directamente:

```
boe.es/trusteando/
```

```
├── identity/
│   ├── public_key/                  ← clave pública del agente
nacional
│   ├── hash_publico/               ← prueba de identidad autónoma
│   └── [boe-ref BOE-A-2027-XXXX]/    ← referencia a la publicación
oficial
├── [acredited-by]/author/         ← acreditado por el autor fundador
├── [state trusteando]/            ← nodo activo y verificado
├── since/2027-01-01/
├── registry/                      ← registro de nodos reconocidos por
España
│   ├── [uma root-domain-agent]/university-malaga/
│   ├── [ucm root-domain-agent]/university-complutense/
│   └── [aeat root-domain-agent]/aeat-root/
```

La referencia [boe-ref BOE-A-2027-XXXX] vincula el nodo digital con el acto jurídico oficial. Un ciudadano, una empresa o un tribunal puede consultar el BOE, encontrar la orden ministerial, y verificar criptográficamente que el nodo en boe.es/trusteando es exactamente el que el Gobierno declaró en esa fecha. La prueba de identidad tiene dos capas independientes: la criptográfica (hash_publico) y la jurídica (publicación en el BOE). Comprometer una no compromete la otra.

Este mismo mecanismo es aplicable a cualquier país con un registro oficial equivalente al BOE: el Diario Oficial de la Unión Europea, el Journal Officiel francés, el Bundesanzeiger alemán. La publicación oficial nacional es el puente entre el sistema jurídico preexistente y el protocolo. No hace falta crear nueva infraestructura legal — basta con usar la que ya existe.

F.6 Ejemplo ilustrativo: suscripción de Estados Unidos mediante el Federal Register (no normativo)

El equivalente americano del BOE es el Federal Register, el diario oficial del gobierno federal de Estados Unidos. Sin embargo, el sistema institucional americano tiene una particularidad relevante para el protocolo: la autoridad está distribuida desde el origen entre agencias federales independientes con competencias técnicas específicas. Esto hace que el modelo americano sea naturalmente policéntrico desde el arranque — en lugar de un único agente nacional, varias agencias pueden actuar como roots de dominio independientes desde el primer día.

```
federalregister.gov/trusteando/
├── identity/
│   ├── public_key/
│   └── [fr-ref FR-2027-XXXXX]/        ← referencia al Federal
Register
├── [acredited-by]/author/
├── [state trusteando]/
├── since/2027-01-01/
├── [quorum 3]/                    ← quórum mínimo entre agencias
└── registry/
    ├── [nist root-domain-agent]/nist-root/     ← estándares técnicos
    ├── [ftc root-domain-agent]/ftc-root/       ← protección del
consumidor
    ├── [ed root-domain-agent]/dept-education-root/  ← credenciales
académicas
    └── [mit root-domain-agent]/mit-root/       ← ejemplo:
instituciones académicas
```

La diferencia estructural respecto al modelo español es ilustrativa de cómo el protocolo se adapta a sistemas institucionales distintos. España centraliza en el BOE y delega hacia abajo. Estados Unidos distribuye desde el origen entre agencias independientes con autoridad propia. En ambos casos el mecanismo de publicación es el mismo — la URL del registro oficial

como ancla jurídica — pero la topología del grafo resultante refleja la arquitectura institucional real de cada país.

Referencias

- ECDH — Elliptic Curve Diffie-Hellman: RFC 7748, Bernstein et al., 2016
- HMAC — Hash-based Message Authentication Code: RFC 2104, Krawczyk et al., 1997
- SHA-256 — Secure Hash Algorithm: FIPS PUB 180-4, NIST, 2015
- W3C Verifiable Credentials Data Model: W3C Recommendation, 2022
- W3C Decentralised Identifiers (DIDs): W3C Recommendation, 2022
- TLS 1.3 — Transport Layer Security: RFC 8446, Rescorla, 2018
- eIDAS — Reglamento (UE) 910/2014 sobre identificación electrónica, 2014
- GNU General Public License v3: Free Software Foundation, 2007

Autor: confidencenode.org/members/confidencenode0 — confidencenode.org/protocolos/trusteando

18 de Marzo de 2026 — v0.1 — ConfidenceNode — confidencenode.org

Trusteando es software libre licenciado bajo GNU GPL v3.
El protocolo es abierto. Cualquiera puede implementarlo, verificarlo, y construir sobre él.
trusteando.app — github.com/ConfidenceNode

Autor: confidencenode.org/members/confidencenode0 — confidencenode.org/protocolos/trusteando
18 de Marzo de 2026 — v0.1 — ConfidenceNode — confidencenode.org
Trusteando es software libre licenciado bajo GNU GPL v3.
El protocolo es abierto. Cualquiera puede implementarlo, verificarlo, y construir sobre él.
trusteando.app — github.com/ConfidenceNode